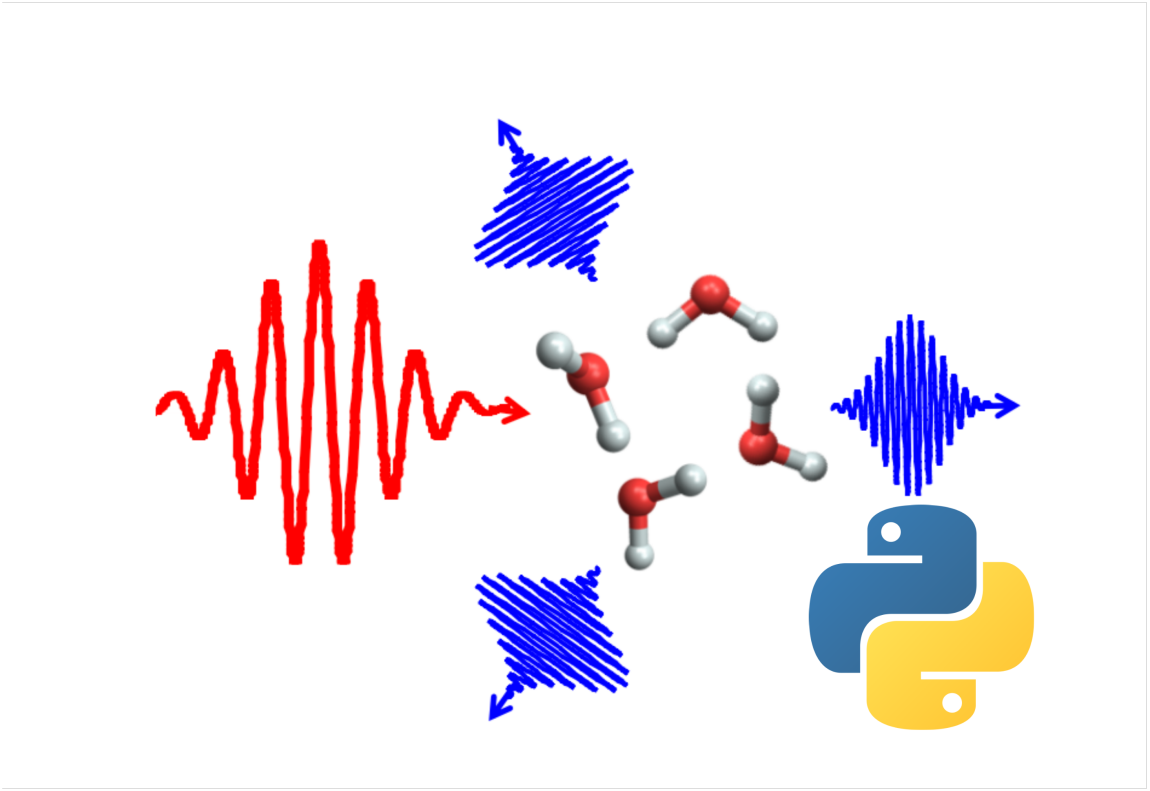
Py_SHS

A Python Package for Second Harmonic Scattering

User Manual — Version 4.0

Pure Python / NumPy, SciPy implementation & Graphical User Interface



Pierre-Marie Gassin
Institut Charles Gerhardt Montpellier – ENSCM
pierre-marie.gassin@enscm.fr

Contents

1 Introduction **4**
- 1.1 What is Py_SHS? . . . 4
- 1.2 Distribution and package layout . . . 5
- 1.3 Installation . . . 5
  - 1.3.1 Requirements (all platforms) . . . 5
  - 1.3.2 Installing Py_SHS with pip . . . 6
  - 1.3.3 Platform notes: tkinter . . . 6
  - 1.3.4 Launching the GUI . . . 7
  - 1.3.5 Installing from the GitHub sources . . . 7
- 1.4 Command-line usage (without GUI) . . . 7

2 General equations implemented in Py_SHS **9**
- 2.1 Notation . . . 9
- 2.2 Incoherent Second Harmonic Scattering (HRS) . . . 10
- 2.3 Coherent SHS of N fully correlated molecules . . . 12
  - 2.3.1 Intensity and effective tensor . . . 12
  - 2.3.2 The three reference frames . . . 13
  - 2.3.3 From β^{mol} to β^{meso} (once per dipole) . . . 13
  - 2.3.4 Coherent sum in the mesoscopic frame: total S mesoscopic tensor . . . 14
  - 2.3.5 Laboratory components and polarisation coefficients . . . 14
  - 2.3.6 Summary of the computational flow . . . 16
- 2.4 Coherent SHS by a sphere . . . 16
  - 2.4.1 Computation with the NLRGD and NLWKB model . . . 16
  - 2.4.2 Computation with the Nonlinear Mie theory . . . 18
- 2.5 Coherent SHS by a cylinder . . . 20
  - 2.5.1 Physical model . . . 20
  - 2.5.2 Geometry and local surface frame . . . 21
  - 2.5.3 Effective nonlinear susceptibility . . . 21
  - 2.5.4 Numerical quadrature and orientational average . . . 22
  - 2.5.5 Polarization-resolved coefficients . . . 22
- 2.6 Coherent SHS by an ellipsoid of revolution . . . 23
  - 2.6.1 Physical model . . . 23
  - 2.6.2 Geometry and local surface frame . . . 23
  - 2.6.3 Effective susceptibility and orientational average . . . 24
- 2.7 Coherent SHS by a rectangular box (cube) . . . 25
  - 2.7.1 Physical model and geometry . . . 25
  - 2.7.2 Effective susceptibility . . . 25
- 2.8 Numerical conventions common to all modules . . . 26
  - 2.8.1 Symmetry of the tensor . . . 26
  - 2.8.2 Orientation quadrature . . . 26
  - 2.8.3 Automatic size of the grids . . . 26
  - 2.8.4 Intensity units . . . 27

2.8.5 Scattering-angle grid . . . 27
2.8.6 Absorbing media and origin . . . 27
2.8.7 Chiral cross terms and the legacy mode . . . 27

3 HRS program **28**
3.1 Inputs and outputs . . . 28
3.2 Command line . . . 28
3.3 Example . . . 29
3.4 Code architecture . . . 30
3.4.1 Low-level utilities . . . 30
3.4.2 Orientation and angle tables . . . 30
3.4.3 Rotation, combination and finalisation . . . 30
3.4.4 Numerical parameters . . . 31
3.4.5 Outputs . . . 31
3.4.6 Code ↔ equation map . . . 31
3.4.7 Limitations and practical notes . . . 31

4 SHS program **32**
4.1 Inputs . . . 32
4.2 Command line . . . 32
4.3 Example: a spherical shell of dipoles . . . 33
4.4 Code architecture . . . 33
4.4.1 Low-level utilities . . . 34
4.4.2 Orientation and angle tables . . . 34
4.4.3 Accumulation and finalisation . . . 34
4.4.4 Numerical parameters . . . 35
4.4.5 Outputs . . . 35
4.4.6 Code ↔ equation map . . . 35
4.4.7 Limitations and practical notes . . . 36

5 Sphere Object (`sphere_SHS.py`) **37**
5.1 Inputs . . . 37
5.2 Command line . . . 37
5.3 Example . . . 38
5.4 Code architecture . . . 39
5.4.1 Low-level utilities . . . 39
5.4.2 Surface integral and coefficients . . . 39
5.4.3 Numerical parameters . . . 39
5.4.4 Outputs . . . 40
5.4.5 Code ↔ equation map . . . 40
5.4.6 Limitations and practical notes . . . 40

6 Cylinder Object (`cylinder_SHS.py`) **41**
6.1 Inputs . . . 41
6.2 Command line . . . 41
6.3 Example . . . 41
6.4 Code architecture . . . 42
6.4.1 Functions . . . 42
6.4.2 Numerical parameters . . . 43
6.4.3 Outputs . . . 43
6.4.4 Code ↔ equation map . . . 43
6.4.5 Limitations and practical notes . . . 43

7 Ellipsoid Object (`ellipsoid_SHS.py`) 44
7.1 Inputs . . . 44
7.2 Command line . . . 44
7.3 Example . . . 44
7.4 Code architecture . . . 45
7.4.1 Functions . . . 45
7.4.2 Numerical parameters . . . 46
7.4.3 Outputs . . . 46
7.4.4 Code ↔ equation map . . . 46
7.4.5 Limitations and practical notes . . . 46

8 Cube / Box Object (`cube_SHS.py`) 47
8.1 Inputs . . . 47
8.2 Command line . . . 47
8.3 Example . . . 47
8.4 Code architecture . . . 48
8.4.1 Functions . . . 48
8.4.2 Numerical parameters . . . 49
8.4.3 Outputs . . . 49
8.4.4 Code ↔ equation map . . . 49
8.4.5 Limitations and practical notes . . . 49

9 Graphical User Interface 50
9.1 Launch . . . 50
9.2 Common layout . . . 51
9.3 HRS tab . . . 51
9.4 SHS tab . . . 51
9.5 Sphere, Cylinder, Ellipsoid and Cube tabs . . . 52
9.6 Results and export . . . 52

10 Additional scripts 53
10.1 Orientation / position generators (SHS) . . . 53
10.2 Gaussian → PySHS tensor . . . 54
10.3 Plotting . . . 54
10.4 Comparing two output files . . . 54
10.5 Where the side files go . . . 55

11 Changes in Version 4 56
11.1 Architecture . . . 56
11.2 Numerical changes that can modify results . . . 56
11.3 New features . . . 57
11.4 Equivalence with the former C programs . . . 57

Chapter 1

Introduction

1.1 What is Py_SHS?

Py_SHS evaluates the Second-Harmonic Scattered (SHS) intensity for several classes of systems. Version 4 is a **pure Python / NumPy** rewrite of the original C cores: the same physics and the same input/output protocols are used, without any C compilation.

Module	Python script	Physical situation
HRS	`HRS.py`	Incoherent SHS of uncorrelated molecules (Hyper-Rayleigh Scattering)
SHS	`SHS.py`	Coherent SHS of N fully correlated molecules (aggregate)
Sphere Object	`sphere_SHS.py`	Continuum surface response of a colloidal sphere under the nonlinear Wentzel–Kramers–Brillouin (NLWKB) model or the Rayleigh Gans Debye (RGD) model
Sphere Mie	`shs_mie.py`	Nonlinear Mie theory for spheres
Cylinder Object	`cylinder_SHS.py`	Continuum surface response of a colloidal cylinder under (NLWKB/RGD) model
Ellipsoid Object	`ellipsoid_SHS.py`	Spheroid / ellipsoid of revolution under (NLWKB/RGD) model
Cube / Box Object	`cube_SHS.py`	Rectangular box or cube (NLWKB / RGD)

For each continuum module three calculation modes (keywords) are available:

`polarplot_single`
: Polarization-resolved intensity $I_V(\gamma)$, $I_H(\gamma)$ at one fixed scattering angle Θ.

`polarplot_integrate`
: Same polarization plot, coefficients averaged between two scattering angles.

`angle_scattering`
: Intensity versus scattering angle for fixed polarization combinations IV(0°), IV(90°), IH(0°), IH(90°).

1.2 Distribution and package layout

Py_SHS is distributed on the Python Package Index (PyPI) under the name `pyshs-optics` and is installed with `pip` (Section 1.3). Nothing has to be unpacked, compiled or copied into a particular directory. The installation provides:

- the Python package `py_shs` (the calculation modules, the GUI, the plotting and generation scripts), installed in the `site-packages` directory of the Python environment used for the installation;
- a set of console commands (`pyshs-optics-gui`, `pyshs-optics-hrs`, ...) added to the `PATH` of that environment (Table 1.1).

The installed package has the following content (you normally never need to open it):

```
py_shs/                          # in site-packages, managed by pip
|-- pyshs_gui.py                 # Tkinter GUI  (command pyshs-optics-gui)
|-- HRS.py, SHS.py
|-- sphere_SHS.py, cylinder_SHS.py
|-- ellipsoid_SHS.py, cube_SHS.py
|-- shs_mie.py                   # nonlinear Mie theory
|-- graph_angle.py, graph_assembly.py
|-- script_*.py                  # orientation / tensor generators
|-- shs_grid.py, shs_units.py, shs_angles.py, shs_io.py
|-- cli.py                       # entry points of the pyshs-optics-* commands
|-- __main__.py                  # python -m py_shs  (starts the GUI)
`-- __init__.py
```

No `Work/`, `Src/` or `run_gui.py` directory. Earlier distributions of Py_SHS were zip archives containing a `Src/` directory, a `run_gui.py` launcher and a `Work/` directory for the input and output files. With the pip installation:

- the source code is inside the installed package (there is no `Src/` directory to find or to run from): the programs are started with the console commands, or with `python -m py_shs.<module>`, from any directory;
- `run_gui.py` is replaced by the command `pyshs-optics-gui` (or `python -m py_shs`);
- **there is no `Work/` directory in the installation.** The package directory belongs to `pip`: it may be read-only and it is replaced at every update, so it is not a place to store results. Your files are kept elsewhere:
  - the **GUI** writes its input and output files in a directory `PySHS_Work` created in your home folder at the first launch (Section 9.1); another location can be chosen with the environment variable `PYSHS_WORK`;
  - the **command-line programs** use no fixed directory: input and output files are where you put them (the current directory, or the paths given on the command line, Section 1.4).

This manual is not part of the pip package: it is available in the `Documentation/` directory of the GitHub repository, https://github.com/pmgassin/Py_SHS.

1.3 Installation

1.3.1 Requirements (all platforms)

- Python 3.8 or newer, from https://www.python.org or from your system (`pip` is included in current Python distributions);

- `numpy`, `scipy` (nonlinear Mie module) and `matplotlib`: **installed automatically** by `pip`;
- for the GUI only: `tkinter`, which is part of the standard Python installers of Windows and macOS but is a separate system package on Linux (see the platform notes below). It cannot be installed with `pip`.

A C compiler is **not** required.

1.3.2 Installing Py_SHS with pip

Open a terminal (Command Prompt or PowerShell on Windows) and type:

```
pip install pyshs-optics
```

If `pip` is not found, or to be sure that it belongs to the Python you intend to use, write `python3 -m pip install pyshs-optics` (`python -m pip ...` on Windows). To check the installation:

```
python3 -c "import py_shs; print(py_shs.__version__)"
pip show pyshs-optics
```

It is recommended to install Py_SHS in a *virtual environment*, which keeps it and its dependencies separate from the rest of your Python installation (the commands are then available as long as the environment is activated):

```
python3 -m venv pyshs-env
source pyshs-env/bin/activate          # Windows: pyshs-env\Scripts\activate
pip install pyshs-optics
```

With Anaconda or Miniconda, activate your conda environment and run the same `pip install` command inside it.

To update or to remove the package:

```
pip install --upgrade pyshs-optics
pip uninstall pyshs-optics
```

Uninstalling does not delete the `PySHS_Work` directory of the GUI, which contains your own files.

1.3.3 Platform notes: tkinter

The command-line programs do not need `tkinter`. If the GUI fails to start with the message `No module named 'tkinter'`, install it for your system:

```
# Debian / Ubuntu
sudo apt update
sudo apt install python3-tk

# Fedora
sudo dnf install python3-tkinter

# macOS with a Homebrew Python (the python.org installer already includes tkinter)
brew install python-tk
```

On Windows, install Python from https://www.python.org, ticking "tcl/tk and IDLE" and "Add Python to PATH"; to add tkinter to an existing installation, run the installer again and choose "Modify".

1.3.4 Launching the GUI

```
pyshs-optics-gui
```

or, equivalently, `python3 -m py_shs` (`python -m py_shs` on Windows). If the command `pyshs-optics-gui` is reported as not found, the environment in which Py_SHS was installed is not activated, or the directory where `pip` puts the commands is not in your `PATH`; `python -m py_shs` always works. The GUI is described in Chapter 9.

1.3.5 Installing from the GitHub sources

To work on the source code or to use the development version, clone the repository and install it from the local directory:

```
git clone -b Py_SHS_V4 https://github.com/pmgassin/Py_SHS.git
cd Py_SHS
pip install .
```

1.4 Command-line usage (without GUI)

Each module is a console command installed by `pip` (Table 1.1). It can also be started with the Python interpreter, from any directory, with `python3 -m py_shs.<module>`; the two forms are strictly equivalent. In the rest of this manual the commands are written in their `pyshs-optics-*` form.

Module	Console command	Equivalent
GUI	`pyshs-optics-gui`	`python3 -m py_shs`
HRS	`pyshs-optics-hrs`	`python3 -m py_shs.HRS`
SHS (assembly)	`pyshs-optics-shs`	`python3 -m py_shs.SHS`
Sphere Object	`pyshs-optics-sphere`	`python3 -m py_shs.sphere_SHS`
Sphere Mie	`pyshs-optics-mie`	`python3 -m py_shs.shs_mie`
Cylinder Object	`pyshs-optics-cylinder`	`python3 -m py_shs.cylinder_SHS`
Ellipsoid Object	`pyshs-optics-ellipsoid`	`python3 -m py_shs.ellipsoid_SHS`
Cube / Box Object	`pyshs-optics-cube`	`python3 -m py_shs.cube_SHS`

Table 1.1: Console commands installed with the package `pyshs-optics`.

Parameters that used to be typed interactively for the C binaries are still read from **standard input**: the examples below pipe the answers, and a command run alone prompts for them.

```
# HRS
pyshs-optics-hrs <keyword> <input_beta> <output_file>

# SHS (two input files)
pyshs-optics-shs <keyword> <input_beta> <input_orientation> <output_file>

# Sphere / Cylinder / Ellipsoid / Cube
pyshs-optics-sphere     <keyword> <input_beta> <output_file>
pyshs-optics-cylinder   <keyword> <input_beta> <output_file>
pyshs-optics-ellipsoid  <keyword> <input_beta> <output_file>
pyshs-optics-cube       <keyword> <input_beta> <output_file>

# Nonlinear Mie (sphere)
pyshs-optics-mie        <keyword> <input_beta> <output_file>
```

where `<keyword>` is one of `polarplot_single`, `polarplot_integrate`, `angle_scattering`. The input files and the output file are given as ordinary paths: the programs read and write exactly the files you name, in the current directory or elsewhere.

Examples with piped answers (Linux/macOS shell, or Git Bash/WSL on Windows):

```
# HRS polar plot at 90 deg
printf "800\n1.33\n0.0\n90\n" | \
  pyshs-optics-hrs polarplot_single beta.txt out_hrs.txt

# Sphere NLWKB at 90 deg
printf "1\n800\n1.5\n0.0\n1.33\n0.0\n50\n90\n" | \
  pyshs-optics-sphere polarplot_single beta.txt out_sph.txt
```

Environment variables:

Variable	Effect
`PYSHS_WORK=<dir>`	Working directory of the *GUI* (default: `PySHS_Work` in the home folder, Section 9.1); not used by the command-line programs
`PYSHS_THREADS=`k	Thread count for HRS / SHS (default: number of CPU cores)
`PYSHS_LEGACY=1`	Reproduce two historical normalisation quirks of the old C HRS/SHS codes
`PYSHS_I_SCALE=`x	Optional intensity scale factor, applied by every module

The variables that control the units, the angle grid and the quadrature grids (`PYSHS_UNITS`, `PYSHS_ANGLE_RANGE`, `PYSHS_GRID`, ...) are described in Sections 2.8.4, 2.8.5 and 2.8.3. On Linux and macOS a variable is set for one command by writing it before the command (`PYSHS_LEGACY=1 pyshs-optics-hrs ...`) or for the whole session with `export`; on Windows use `set` (Command Prompt) or `$env:` (PowerShell).

Plotting helpers (the image is written where you indicate):

```
python3 -m py_shs.graph_angle    out_plot image.png
python3 -m py_shs.graph_assembly orientation_file image.png
```

Chapter 2

General equations implemented in Py_SHS

2.1 Notation

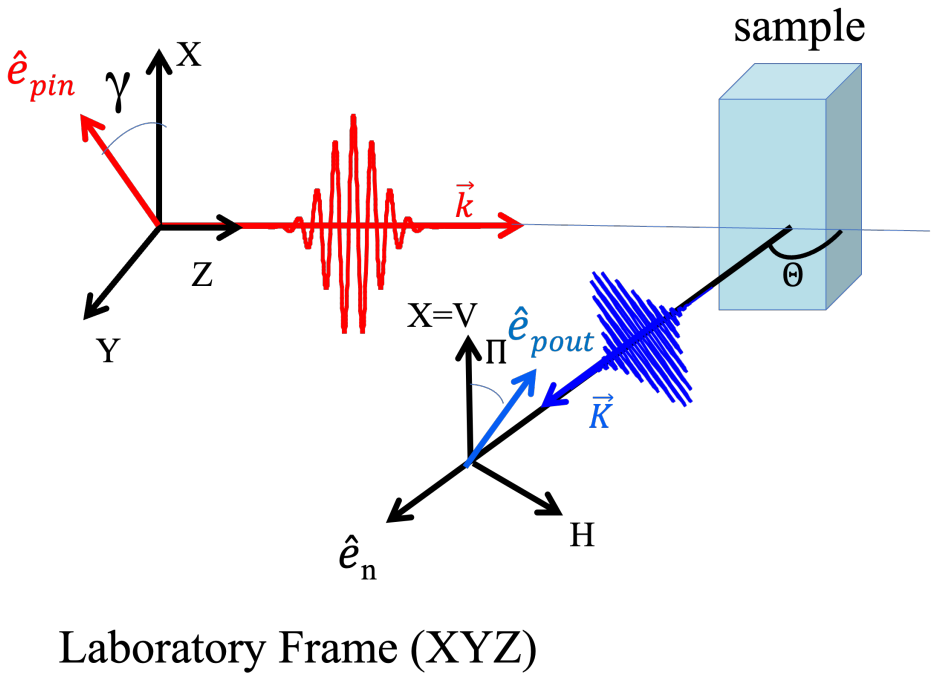


Figure 2.1: Laboratory frame: fundamental wave vector $\mathbf{k}^{\omega}$ and scattered wave vector $\mathbf{K}^{2\omega}$ at angle Θ .

The fundamental field at frequency ω is characterized by its wave vector $k^{\omega}.\mathbf{e}_Z$ and polarization direction $\mathbf{e}^{\omega}_{p-in}$. The input polarization angle γ is the angle between $\mathbf{e}^{\omega}_{p-in}$ and the laboratory X axis. The second-harmonic field at 2ω is characterized by $K^{2\omega}.\mathbf{e}_n$ and the scattering angle Θ. The output polarization angle Π defines the detected polarization $\mathbf{e}^{2\omega}_{p-out}$. In the laboratory frame, the following unit vectors are defined by:

$$\mathbf{e}^{2\omega}_{n} = (0, \sin\Theta,\ \cos\Theta) \tag{2.1}$$

$$\mathbf{e}^{\omega}_{p-in} = (\cos\gamma,\ \sin\gamma,\ 0) \tag{2.2}$$

$$\mathbf{e}^{2\omega}_{p-out} = (\cos\Pi,\ \sin\Pi\cos\Theta,\ \sin\Pi\sin\Theta) \tag{2.3}$$

The scattered wave vector is defined by:

$$\Delta\mathbf{k} = 2\mathbf{k}^{\omega} - \mathbf{K}^{2\omega} = (0, -K\sin\Theta, 2k - K\cos\Theta). \tag{2.4}$$

2.2 Incoherent Second Harmonic Scattering (HRS)

The medium is treated as homogeneous with a single (possibly complex) refractive index n. Wavevector magnitudes are

$$k^{\omega} = \frac{2\pi n}{\lambda}, \qquad k^{2\omega} = \frac{4\pi n}{\lambda}. \tag{2.5}$$

The Second Harmonic Scattering of uncorrelated molecule in solution is usually referred to Hyper Rayleigh Scattering (HRS) and can be expressed as:

$$I_{HRS}(\mathbf{e}^{\omega}_{p-in}, \mathbf{e}^{2\omega}_{p-out}, \mathbf{e}^{2\omega}_{n}) = I_{HRS}(\gamma, \Pi, \Theta) =< \boldsymbol{\beta}_{eff}(\gamma, \Pi, \Theta).\boldsymbol{\beta}^{*}_{eff}(\gamma, \Pi, \Theta) > \tag{2.6}$$

Here

$$\boldsymbol{\beta_{eff}} = \mathbf{e}^{2\omega}_{n} \times (\overleftrightarrow{\beta} : \mathbf{e}^{\omega}_{p-in}\mathbf{e}^{\omega}_{p-in}) \times \mathbf{e}^{2\omega}_{n} \tag{2.7}$$

where $\overleftrightarrow{\beta}$ is the second order hyperpolarisability of the molecule expressed in the laboratory frame. The link between the expression of $\overleftrightarrow{\beta}$ in the laboratory and microscopic frames is:

$$\beta^{\text{lab}}_{IJK} = \sum_{i=1}^{3}\sum_{j=1}^{3}\sum_{k=1}^{3} T_{Ii}T_{Jj}T_{Kk}\,\beta^{\text{micro}}_{ijk}, \tag{2.8}$$

where β^{lab}_{IJK} is the molecular hyperpolarizability expressed in the laboratory frame and $\beta^{\text{micro}}_{ijk}$ is the molecular hyperpolarizability expressed in the molecular frame. Transformation of the molecular frame uses the elements of the rotation matrix $T_{ab}(\varphi, \theta, \psi)$ using the ZYZ convention:

$$T(\varphi,\theta,\psi) = \begin{pmatrix} \cos\theta\cos\varphi\cos\psi - \sin\varphi\sin\psi & -\cos\psi\sin\varphi - \cos\theta\cos\varphi\sin\psi & \cos\varphi\sin\theta \\ \cos\theta\cos\psi\sin\varphi + \cos\varphi\sin\psi & \cos\varphi\cos\psi - \cos\theta\sin\varphi\sin\psi & \sin\theta\sin\varphi \\ -\cos\psi\sin\theta & \sin\theta\sin\psi & \cos\theta \end{pmatrix} \tag{2.9}$$

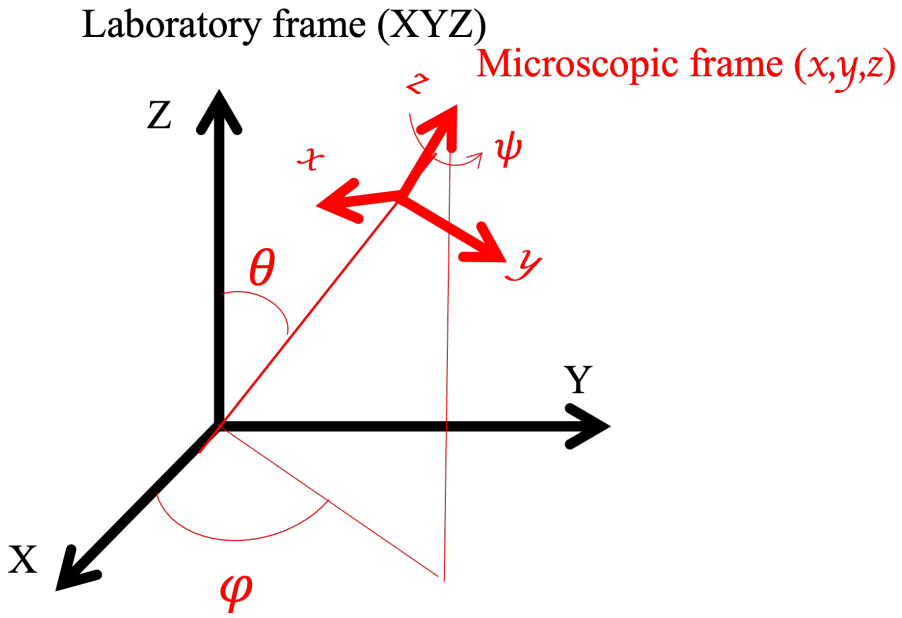


Figure 2.2: Euler angle (φ, θ, ψ) relating the molecular frame to the laboratory frame (ZYZ convention).

Finally, the bracket $\langle\cdot\rangle$ denote the isotropic orientational average.

$$\langle f(\varphi,\theta,\psi)\rangle = \frac{\int_{\psi=0}^{2\pi}\int_{\theta=0}^{\pi}\int_{\varphi=0}^{2\pi} f(\varphi,\theta,\psi)\sin\theta\, d\varphi d\theta\, d\psi}{\int_{\psi=0}^{2\pi}\int_{\theta=0}^{\pi}\int_{\varphi=0}^{2\pi}\sin\theta\, d\varphi d\theta\, d\psi} \tag{2.10}$$

Expanding the equation (2.6) yields the following expression of the effective hyperpolarizability expressed in term of the tensor components expressed in the laboratory frame:

$$\boldsymbol{\beta_{eff}}(\gamma,\Theta) = \begin{bmatrix} \beta_{XXX}\cos^2\gamma + 2\beta_{XXY}\cos\gamma\sin\gamma + \beta_{XYY}\sin^2\gamma \\ \left(\beta_{YYY}\cos^2\gamma + 2\beta_{YXY}\cos\gamma\sin\gamma + \beta_{YYY}\sin^2\gamma\right)\cos^2\Theta \\ -\left(\beta_{ZYY}\cos^2\gamma + 2\beta_{ZXY}\cos\gamma\sin\gamma + \beta_{ZYY}\sin^2\gamma\right)\cos\Theta\sin\Theta \\ -\left(\beta_{YYY}\cos^2\gamma + 2\beta_{YXY}\cos\gamma\sin\gamma + \beta_{YYY}\sin^2\gamma\right)\cos\Theta\sin\Theta \\ +\left(\beta_{ZYY}\cos^2\gamma + 2\beta_{ZXY}\cos\gamma\sin\gamma + \beta_{ZYY}\sin^2\gamma\right)\sin^2\Theta \end{bmatrix} \tag{2.11}$$

The equations (2.6) and (2.11) can be expressed as polarization resolved plots:

$$I_V(\gamma,\Theta) = a_V^\Theta\cos^4\gamma + b_V^\Theta\cos^2\gamma\sin^2\gamma + c_V^\Theta\sin^4\gamma, \tag{2.12}$$

$$I_H(\gamma,,\Theta) = a_H^\Theta\cos^4\gamma + b_H^\Theta\cos^2\gamma\sin^2\gamma + c_H^\Theta\sin^4\gamma. \tag{2.13}$$

The six coefficients $a_V^\Theta, b_V^\Theta, c_V^\Theta, a_H^\Theta, b_H^\Theta, c_H^\Theta$ are combinations of orientationally averaged products of laboratory-frame β components according to:

$$a_V^\Theta = \langle\beta_{XXX}\beta^*_{XXX}\rangle \tag{2.14}$$

$$c_V^\Theta = \langle\beta_{XYY}\beta^*_{XYY}\rangle \tag{2.15}$$

$$b_V^\Theta = \langle 4\beta_{XXY}\beta^*_{XXY} + \beta_{XXX}\beta^*_{XYY} + \beta^*_{XXX}\beta_{XYY}\rangle \tag{2.16}$$

$$a_H^\Theta = \langle\beta_{YXX}\beta^*_{YXX}\rangle\cos^2\Theta + \langle\beta_{ZXX}\beta^*_{ZXX}\rangle\sin^2\Theta - \langle\beta_{YXX}\beta^*_{ZXX} + \beta_{ZXX}\beta^*_{YXX}\rangle\cos\Theta\sin\Theta \tag{2.17}$$

$$c_H^\Theta = \langle\beta_{YYY}\beta_{YYY}\rangle^*\cos^2\Theta + \langle\beta_{ZYY}\beta^*_{ZYY}\rangle\sin^2\Theta - \langle\beta_{YYY}\beta^*_{ZYY} + \beta_{ZYY}\beta^*_{YYY}\rangle\cos\Theta\sin\Theta \tag{2.18}$$

$$\begin{aligned} b_H^\Theta = {} & \left\langle 4\beta_{ZXY}\beta^*_{ZXY} + \beta_{ZXX}\beta^*_{ZYY} + \beta^*_{ZXX}\beta_{ZYY}\right\rangle\sin^2\Theta \\ & + \left\langle 4\beta_{YYX}\beta^*_{YYX} + \beta_{YXX}\beta^*_{YYY} + \beta^*_{YXX}\beta_{YYY}\right\rangle\cos^2\Theta \\ & - \cos\Theta\sin\Theta\big\langle 4\beta_{YYX}\beta^*_{ZXY} + 4\beta_{ZXY}\beta^*_{YYX} + \beta_{ZXX}\beta^*_{YYY} + \beta^*_{ZXX}\beta_{YYY} \\ & \qquad + \beta_{YXX}\beta^*_{ZYY} + \beta_{ZYY}\beta^*_{YXX}\big\rangle \end{aligned} \tag{2.19}$$

Additional dimensionless ratios $I_2^{\Theta,\Pi}$ and $I_4^{\Theta,\Pi}$ can also be computed:

$$I_2^{\Theta,\Pi} = \frac{4(a_\Pi^\Theta - c_\Pi^\Theta)}{3a_\Pi^\Theta + b_\Pi^\Theta + 3c_\Pi^\Theta}, \qquad I_4^{\Theta,\Pi} = \frac{a_\Pi^\Theta - b_\Pi^\Theta + c_\Pi^\Theta}{3a_\Pi^\Theta + b_\Pi^\Theta + 3c_\Pi^\Theta}, \tag{2.20}$$

$$\tag{2.21}$$

In `angle_scattering` mode the program evaluates the following quantities as functions of the scattering angle Θ.

$$I_V(0^\circ) = I_{HRS}(\gamma=0,\Pi=0,\Theta) = a_V^\Theta; \tag{2.22}$$

$$I_V(90^\circ) = I_{HRS}(\gamma=90,\Pi=0,\Theta) = c_V^\Theta, \tag{2.23}$$

$$I_H(0^\circ) = I_{HRS}(\gamma=0,\Pi=90,\Theta) = a_H^\Theta, \tag{2.24}$$

$$I_H(90^\circ) = I_{HRS}(\gamma=90,\Pi=90,\Theta) = c_H^\Theta \tag{2.25}$$

The program can also report the isotropic HRS hyperpolarizability defined for $\Theta = 90$:

$$\beta_{\mathrm{HRS}} = \sqrt{\langle \beta^2_{XXX} \rangle + \langle \beta^2_{ZXX} \rangle} \tag{2.26}$$

and the depolarization ratio

$$D = \frac{c_V^{\Theta}}{a_V^{\Theta}} = \frac{\langle \beta^2_{XYY} \rangle}{\langle \beta^2_{XXX} \rangle}. \tag{2.27}$$

2.3 Coherent SHS of N fully correlated molecules

The module `SHS.py` computes the **coherent** second-harmonic scattered intensity of an aggregate of N molecules (or dipoles) that are **fully correlated** in position and orientation. It sits between incoherent HRS (N independent molecules) and the continuum surface models (Sphere, Cylinder, ...).

2.3.1 Intensity and effective tensor

In a homogeneous medium of refractive index n the coherent intensity is

$$I_{\mathrm{SHS}}(\gamma, \Pi, \Theta) = \Big\langle \beta_{\mathrm{eff},t}(\gamma, \Pi, \Theta)\, \beta^*_{\mathrm{eff},t}(\gamma, \Pi, \Theta) \Big\rangle, \tag{2.28}$$

where the contracted tensor (this is a vector) $\beta_{eff,t}$, expressed in the laboratory frame is defined according to

$$\beta_{eff,t} = \mathbf{e}_n^{2\omega} \times \big(\overleftrightarrow{\beta}_{total,XYZ} : \mathbf{e}^{\omega}_{p\text{-in}} \mathbf{e}^{\omega}_{p\text{-in}} \big) \times \mathbf{e}_n^{2\omega} \tag{2.29}$$

and the total assembly tensor is the coherent sum of the individual tensor

$$\overleftrightarrow{\beta}_{total} = \sum_{l=1}^{N} \overleftrightarrow{\beta}_{\mathbf{l}}\, e^{i\, \Delta\mathbf{k}\cdot\mathbf{r}_l}. \tag{2.30}$$

Starting from the positions $\mathbf{r}_l$ and the orientations $\Omega'_l = (\varphi'_l, \theta'_l, \psi'_l)$ of the N dipoles that form the aggregate (all given in the **mesoscopic** frame of the assembly), the program first builds a total hyperpolarizability $\beta^{meso}_{x'y'z'}$ expressed in the mesoscopic frame. This tensor depends on the orientation of the mesoscopic frame in the Laboratory $\Omega = (\varphi, \theta, \psi)$ as depicted in Figure 2.3

$$\overleftrightarrow{\beta}^{\mathrm{meso}}_{total}(\Omega, \Theta) = \sum_{l=1}^{N} \overleftrightarrow{\beta}^{\mathrm{meso}}_{l}\, e^{i\, \mathbf{q}(\Omega,\Theta)\cdot\mathbf{r'}_l}. \tag{2.31}$$

where $\mathbf{q}(\Omega, \Theta)$ is the scattered wave vector expressed in the mesoscopic frame.

$$\mathbf{q} \equiv \Delta\mathbf{k}^{\mathrm{meso}} = T^{\mathsf{T}} \Delta\mathbf{k}^{\mathrm{Labo}} \tag{E}$$

A laboratory vector is expressed in the mesoscopic frame by using the T matrix transpose:T^{T}. The invariance of the scalar product leeds to :

$$\Delta\mathbf{k}^{\mathrm{lab}} \cdot \mathbf{r}_l^{\mathrm{lab}} = \mathbf{q} \cdot \mathbf{r}_l^{\mathrm{meso}}, \tag{2.32}$$

Finally, $\overleftrightarrow{\beta}^{\mathrm{meso}}_{total}(\Omega, \Theta)$ is expressed in the Laboratory frame as $\overleftrightarrow{\beta}^{Labo}_{total}$ and isotropic averages over the whole orientation Ω assembly to deduce the SHS Intensity according to equations 2.28 and 2.29.The subsection below details each step of this chain as it is implemented in the code.

2.3.2 The three reference frames

Frame	Axes	Role
Molecular (microscopic)	x, y, z	Input tensor β^{mol} (file `input_beta`)
Mesoscopic (assembly)	x', y', z'	Relative positions $\mathbf{r}'_l$ and orientations $\Omega' = (\varphi', \theta', \psi')$ of the N dipoles (file `input_orientation`)
Laboratory	X, Y, Z	Input polarisation γ, output polarisation Π, scattering angle Θ

Two ZYZ Euler rotations are involved:

1. Molecule → mesoscopic: angles $(\varphi'_l, \theta'_l, \psi'_l)$ of each dipole (orientation file).
2. Mesoscopic → laboratory: angles (φ, θ, ψ) of the **whole assembly** (isotropic orientational average).

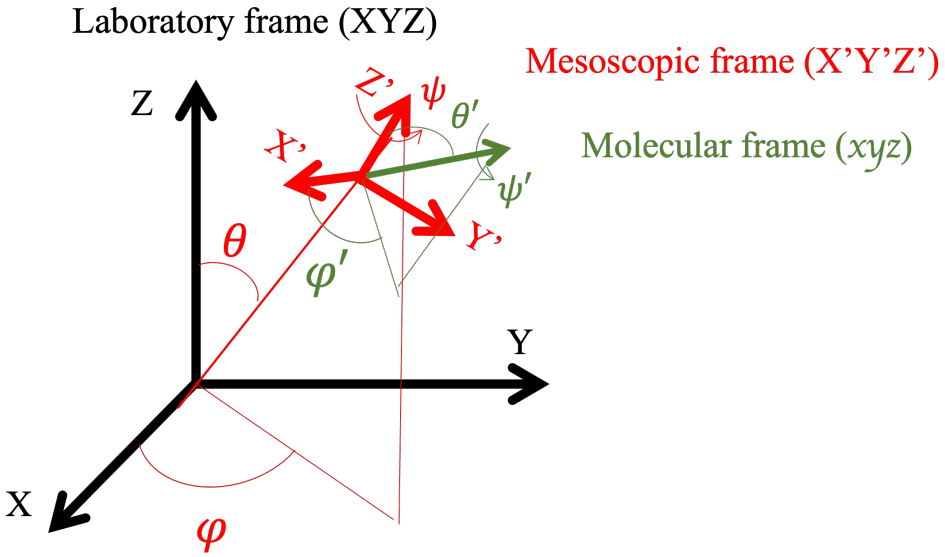


Figure 2.3: Definition of the angles $(\varphi_l, \theta_l, \psi_l)$ and $(\varphi'_l, \theta'_l, \psi'_l)$ that orient the scattered objects in the mesoscopic and laboratory frames.

The ZYZ rotation matrix that maps a mesoscopic vector to the laboratory is

$$T(\varphi, \theta, \psi) = \begin{pmatrix} T_{Xx'} & T_{Xy'} & T_{Xz'} \\ T_{Yx'} & T_{Yy'} & T_{Yz'} \\ T_{Zx'} & T_{Zy'} & T_{Zz'} \end{pmatrix}, \tag{2.33}$$

with the different elements are given by equation 2.9

2.3.3 From β^{mol} to β^{meso} (once per dipole)

For each dipole l the orientation file provides $(\varphi'_l, \theta'_l, \psi'_l)$ with $T^{(l)} = T(\varphi'_l, \theta'_l, \psi'_l)$ one obtains the mesoscopic tensor

$$\beta^{(l),\mathrm{meso}}_{i'j'k'} = \sum_{i,j,k=1}^{3} T^{(l)}_{i'i}\, T^{(l)}_{j'j}\, T^{(l)}_{k'k}\, \beta^{\mathrm{mol}}_{ijk}. \tag{2.34}$$

Here the unprimed indices (i, j, k) run over the **molecular** axes (x, y, z), while the primed indices (i', j', k') run over the **mesoscopic** axes (x', y', z'). Positions $\mathbf{r}_l = (x'_l, y'_l, z'_l)$ are already expressed in the mesoscopic frame.

2.3.4 Coherent sum in the mesoscopic frame: total S mesoscopic tensor

The phase wave vector in the laboratory is

$$\Delta\mathbf{k}^{\text{lab}} = 2\mathbf{k}^{\omega} - \mathbf{K}^{2\omega} = \big(0,\ -K\sin\Theta,\ 2k - K\cos\Theta\big), \tag{2.35}$$

with $k = 2\pi n/\lambda$ and $K = 4\pi n/\lambda$. The code rotates $\Delta\mathbf{k}$ into the mesoscopic frame with assembly orientation $\Omega = (\varphi, \theta, \psi)$,

For a given assembly orientation Ω and scattering angle Θ, the total mesoscopic tensor S is:

$$S_{i'j'k'}(\Omega,\Theta) = \sum_{l=1}^{N} e^{i\,\mathbf{q}(\Omega,\Theta)\cdot\mathbf{r}_l}\ \beta_{i'j'k'}^{(l),\text{meso}}. \tag{2.36}$$

S already contains all spatial interference among the N dipoles.

The S tensor can then be expressed in the Laboratory Frame :

$$A_{XYZ} = S_{XYZ}^{\text{Lab}}(\Omega,\Theta) = \sum_{i',j',k'=1}^{3} T_{Ii'}(\Omega)\,T_{Jj'}(\Omega)\,T_{Kk'}(\Omega)\,S_{i'j'k'}(\Omega,\Theta) \tag{2.37}$$

In absorbing media ($n = n_r + in_i$) one has $\mathbf{q} = \mathbf{q}_r + i\mathbf{q}_i$ and $e^{i\mathbf{q}\cdot\mathbf{r}} = e^{i\mathbf{q}_r\cdot\mathbf{r}}e^{-\mathbf{q}_i\cdot\mathbf{r}}$.

2.3.5 Laboratory components and polarisation coefficients

$$\beta_{eff,t} = \mathbf{e}_n^{2\omega} \times \big(\overset{\leftrightarrow}{\mathbf{A}}_{XYZ} : \mathbf{e}_{p\text{-in}}^{\omega}\mathbf{e}_{p\text{-in}}^{\omega}\big) \times \mathbf{e}_n^{2\omega} \tag{2.38}$$

on pose:

$$\boldsymbol{A} = \begin{bmatrix} A_X \\ A_Y \\ A_Z \end{bmatrix} = \overset{\leftrightarrow}{\mathbf{A}}_{XYZ} : \mathbf{e}_{p\text{-in}}^{\omega}\mathbf{e}_{p\text{-in}}^{\omega} \tag{2.39}$$

$$\boldsymbol{\beta_{eff,t}}(\gamma,\Theta) = \begin{bmatrix} A_X \\ A_Y\cos^2\Theta - A_Z\sin\Theta\cos\Theta \\ A_Z\sin^2\Theta - A_Y\cos\Theta\sin\Theta \end{bmatrix} \tag{2.40}$$

$$= \begin{bmatrix} A_{XXX}\cos^2\gamma + 2A_{XXY}\cos\gamma\sin\gamma + A_{XYY}\sin^2\gamma \\ \big(A_{YYY}\cos^2\gamma + 2A_{YXY}\cos\gamma\sin\gamma + A_{YYY}\sin^2\gamma\big)\cos^2\Theta \\ -\big(A_{ZYY}\cos^2\gamma + 2A_{ZXY}\cos\gamma\sin\gamma + A_{ZYY}\sin^2\gamma\big)\cos\Theta\sin\Theta \\ -\big(A_{YYY}\cos^2\gamma + 2A_{YXY}\cos\gamma\sin\gamma + A_{YYY}\sin^2\gamma\big)\cos\Theta\sin\Theta \\ +\big(A_{ZYY}\cos^2\gamma + 2A_{ZXY}\cos\gamma\sin\gamma + A_{ZYY}\sin^2\gamma\big)\sin^2\Theta \end{bmatrix} \tag{2.41}$$

- Vertical polarisation ($\Pi = 0°$) : $\mathbf{e}_V = (1,0,0)$:

  We define: $\beta_{\text{eff}}^{V}(\gamma) = \mathbf{e}_V \cdot \boldsymbol{\beta}_{\text{eff},t} = A_X$.

$$\beta_V^{eff} = A_{XXX}\cos^2\gamma + 2A_{XXY}\cos\gamma\sin\gamma + A_{XYY}\sin^2\gamma. \tag{2.42}$$

- Horizontal polarisation ($\Pi = 90°$) : $\mathbf{e}_H = (0,\ \cos\Theta,\ -\sin\Theta)$

$$\beta_{\text{eff}}^{H} = \mathbf{e}_H \cdot \boldsymbol{\beta}_{\text{eff},t} \tag{2.43}$$

$$\beta_{\text{eff}}^{H}(\gamma,\Theta) = \cos\Theta A_Y - sin\Theta A_Z \tag{2.44}$$

By defining the quantities:

$$A^H_{XX} = \cos\Theta\, A_{YXX} - \sin\Theta\, A_{ZXX},$$
$$A^H_{YY} = \cos\Theta\, A_{YYY} - \sin\Theta\, A_{ZYY},$$
$$A^H_{XY} = \cos\Theta\, A_{YXY} - \sin\Theta\, A_{ZXY},$$

we can write $\beta^H_{eff}(\gamma,\Theta)$ in the form:

$$\beta^H_{eff} = A^H_{XX}(\Theta)\cos^2\gamma + 2A^H_{XY}(\Theta)\cos\gamma\sin\gamma + A^H_{YY}(\Theta)\sin^2\gamma. \tag{2.45}$$

The equations (2.6) and (2.41) can be expressed as polarization resolved plots, the intensities being then:

$$I_V(\gamma,\Theta) =< \beta^V_{eff}.\beta^V_{eff} >= a_V\cos^4\gamma + b_V\cos^2\gamma\sin^2\gamma + c_V\sin^4\gamma + b_{1V}\cos^3\gamma\sin\gamma + b_{2V}\cos\gamma\sin^3\gamma, \tag{2.46}$$

$$I_H(\gamma,\Theta) =< \beta^H_{eff}.\beta^H_{eff} >= a_H\cos^4\gamma + b_H\cos^2\gamma\sin^2\gamma + c_H\sin^4\gamma + b_{1H}\cos^3\gamma\sin\gamma + b_{2H}\cos\gamma\sin^3\gamma, \tag{2.47}$$

with the following expressions for the ten coefficients $a_V, b_V, b_{1V}, b_{2V}, c_V, a_H, b_H, b_{1H}, b_{2H}, c_H$:

$$a^\Theta_V = \langle A_{XXX}A^*_{XXX}\rangle \tag{2.48}$$

$$c^\Theta_V = \langle A_{XYY}A^*_{XYY}\rangle \tag{2.49}$$

$$b^\Theta_V = \langle 4A_{XXY}A^*_{XXY} + A_{XXX}A^*_{XYY} + A^*_{XXX}A_{XYY}\rangle \tag{2.50}$$

$$b^\Theta_{1V} = \left\langle 2A_{XXX}A^*_{XXY} + 2A^*_{XXX}A_{XXY}\right\rangle \tag{2.51}$$

$$b^\Theta_{2V} = \left\langle 2A_{XXY}A^*_{XYY} + 2A^*_{XXY}A_{XYY}\right\rangle \tag{2.52}$$

$$a^\Theta_H = \langle A_{YXX}A^*_{YXX}\cos^2\rangle\Theta + \langle A_{ZXX}A^*_{ZXX}\rangle\sin^2\Theta - \langle A_{YXX}A^*_{ZXX} + A_{ZXX}A^*_{YXX}\rangle\cos\Theta\sin\Theta \tag{2.53}$$

$$c^\Theta_H = \langle A_{YYY}A^*_{YYY}\cos^2\rangle\Theta + \langle A_{ZYY}A^*_{ZYY}\rangle\sin^2\Theta - \langle A_{YYY}A^*_{ZYY} + A_{ZYY}A^*_{YYY}\rangle\cos\Theta\sin\Theta \tag{2.54}$$

$$\begin{aligned} b^\Theta_H = {} & \left\langle 4A_{ZXY}A^*_{ZXY} + A_{ZXX}A^*_{ZYY} + A^*_{ZXX}A_{ZYY}\right\rangle\sin^2\Theta \\ & + \left\langle 4A_{YYX}A^*_{YYX} + A_{YXX}A^*_{YYY} + A^*_{YXX}A_{YYY}\right\rangle\cos^2\Theta \\ & - \cos\Theta\sin\Theta\big\langle 4A_{YYX}A^*_{ZXY} + 4A_{ZXY}A^*_{YYX} + A_{ZXX}A^*_{YYY} + A^*_{ZXX}A_{YYY} \\ & \qquad + A_{YXX}A^*_{ZYY} + A_{ZYY}A^*_{YXX}\big\rangle \end{aligned} \tag{2.55}$$

$$\begin{aligned} b^\Theta_{1H} = {} & \left\langle 2A_{ZXX}A^*_{ZXY} + 2A^*_{ZXX}A_{ZXY}\right\rangle\sin^2\Theta \\ & + \left\langle 2A_{YXX}\beta^*_{YYX} + 2A^*_{YXX}A_{YYX}\right\rangle\cos^2\Theta \\ & - \cos\Theta\sin\Theta\big\langle 2A_{ZXX}A^*_{ZXY} + 2A_{ZXY}A^*_{ZXX} + 2A_{YXX}A^*_{YXY} + 2A^*_{YXX}A_{YXY}\big\rangle \end{aligned} \tag{2.56}$$

$$\begin{aligned} b^\Theta_{2H} = {} & \left\langle 2A_{ZYY}A^*_{ZXY} + 2A^*_{ZYY}A_{ZXY}\right\rangle\sin^2\Theta \\ & + \left\langle 2A_{YYY}\beta^*_{YYX} + 2A^*_{YYY}A_{YYX}\right\rangle\cos^2\Theta \\ & - \cos\Theta\sin\Theta\big\langle 2A_{ZYY}A^*_{ZXY} + 2A_{ZXY}A^*_{ZYY} + 2A_{YYY}A^*_{YXY} + 2A^*_{YYY}A_{YXY}\big\rangle \end{aligned} \tag{2.57}$$

The isotropic orientational average is

$$\langle f(\varphi,\theta,\psi)\rangle = \frac{1}{8\pi^2}\int_0^{2\pi} d\varphi \int_0^{\pi} \sin\theta\, d\theta \int_0^{2\pi} d\psi\; f(\varphi,\theta,\psi), \tag{2.58}$$

discretised on a $20 \times 10 \times 20$ midpoint grid (4000 points).
The dimensionless ratios

$$I_2^{\Pi} = \frac{4(a_{\Pi}^{\Theta} - c_{\Pi}^{\Theta})}{3a_{\Pi}^{\Theta} + b_{\Pi}^{\Theta} + 3c_{\Pi}^{\Theta}}, \qquad I_4^{\Pi} = \frac{a_{\Pi}^{\Theta} - b_{\Pi}^{\Theta} + c_{\Pi}^{\Theta}}{3a_{\Pi}^{\Theta} + b_{\Pi}^{\Theta} + 3c_{\Pi}^{\Theta}}. \tag{2.59}$$

In `angle_scattering` mode only the extremes $I_V(0^\circ) = a_V^{\Theta} = I_{V_{in}V_{out}}(\Theta)$, $I_V(90^\circ) = c_V\Theta = I_{H_{in}V_{out}}(\Theta)$, $I_H(0^\circ) = a_H^{\Theta} = I_{V_{in}H_{out}}(\Theta)$, $I_H(90^\circ) = c_H^{\Theta} = I_{H_{in}H_{out}}(\Theta)$ are retained.

Remark. A coherent calculation with $N = 1$ recovers exactly the HRS numerical result (the phase factor becomes a global constant).

2.3.6 Summary of the computational flow

β_{mol} (file)
  $\downarrow$ $T(\varphi_l', \theta_l', \psi_l')$ (once per dipole)
$\beta_{\text{meso}}^{(l)} = P_l$ (N tensors $3 \times 3 \times 3$)
  $\downarrow$ $\sum_l \exp(i\,\mathbf{q}(\Omega)\cdot\mathbf{r}_l)\,P_l$
$S_{i'j'k'}(\Omega,\Theta)$ (total mesoscopic phased tensor on x', y', z')
  $\downarrow$ $T(\varphi,\theta,\psi)$ (assembly rotation $\rightarrow$ lab X, Y, Z)
$\beta_{IJK}^{\text{lab}}(\Omega,\Theta)$
  $\downarrow$ contraction (XX, YY, XY)
$A_{I,q}$ $(I = X, Y, Z;\ q = XX, YY, XY)$
  $\downarrow$ V / H combination according to Θ
$\beta_V^{\text{eff}}(\gamma)$, $\beta_H^{\text{eff}}(\gamma)$
  $\downarrow$ $|\cdot|^2$ + average $\langle\cdot\rangle_\Omega$
a, b, c, b_1, b_2
  $\downarrow$
  $I_V(\gamma,\Theta)$, $I_H(\gamma,\Theta)$

2.4 Coherent SHS by a sphere

2.4.1 Computation with the NLRGD and NLWKB model

Two kind of calculation can be performed:

- The Non Linear Rayleigh Gans Debye (NLRGD) model. In this model, only one complex refractive index is considered.
- The NonLinear Wentzel–Kramers–Brillouin model (NLWKB). In this model, the medium is considered as inhomogeneous and two complex refractive indices are used, n_p, the refractive index of the *particle*, and n_m, the refractive index of the surrounding *medium*.

In the Nonlinear Wentzel–Kramers–Brillouin (NLWKB) approximation the wave-vector magnitudes are

$$k^{\omega} = \frac{2\pi n_p}{\lambda} \qquad \text{(inside the particle)}, \tag{2.60}$$

$$K^{2\omega} = \frac{4\pi n_m}{\lambda} \qquad \text{(in the medium)}. \tag{2.61}$$

The classical Rayleigh–Gans–Debye limit is recovered by setting $n_p = n_m = n$.

The intensity is obtained from an effective nonlinear susceptibility tensor $\overset{\leftrightarrow}{\Gamma}^{(2)}_{ijk}$ representing the response of the whole particle surface [1]:

$$I_{SHS}(\mathbf{e}^{\omega}_{p-in}, \mathbf{e}^{2\omega}_{p-out}, \mathbf{e}^{2\omega}_{n}) = I_{SHS}(\gamma, \Pi, \Theta) = \mathbf{\Gamma}_{eff}(\gamma, \Pi, \Theta).\mathbf{\Gamma}^{*}_{eff}(\gamma, \Pi, \Theta) \tag{2.62}$$

where $\mathbf{\Gamma}_{eff}$ is defined as:

$$\mathbf{\Gamma}_{eff} = \mathbf{e}^{2\omega}_{n} \times (\overset{\leftrightarrow}{\mathbf{\Gamma}^{(2)}} : \mathbf{e}^{\omega}_{p-in}\mathbf{e}^{\omega}_{p-in}) \times \mathbf{e}^{2\omega}_{n} \tag{2.63}$$

here

$$\overset{\leftrightarrow}{\mathbf{\Gamma}^{(2)}} = \iint_{surface} \overset{\leftrightarrow}{\chi}^{(2)} e^{i\Delta\mathbf{k}.\mathbf{r}} dS \tag{2.64}$$

where $\overset{\leftrightarrow}{\chi}^{(2)}$ is the surface second order nonlinear susceptibility tensor expressed in the particle frame (x',y',z').

For a sphere of radius R,

$$\Gamma^{(2)}_{i'j'k'}(\Theta) = \int_0^{2\pi} \mathrm{d}\varphi_c \int_0^{\pi} \mathrm{d}\theta_c \, R^2 \sin\theta_c \, \chi^{(2)}_{i'j'k'}(\varphi_c, \theta_c) \, e^{i\,\Phi(\varphi_c, \theta_c, \Theta)}, \tag{2.65}$$

where

$$\varphi(\phi_c, \theta_c, \Theta) = (-2k^{\omega} + K^{2\omega}\cos\Theta)\, R\cos\theta_c + K^{2\omega}\sin\Theta\, R\sin\theta_c \sin\phi_c \tag{2.66}$$

accounts for optical path differences.

Here $\chi^{(2)}_{i'j'k'}$ is the surface susceptibility expressed in the particle frame which can be deduced from the surface susceptibility expressed in the surface frame by:

$$\chi^{(2)}_{i'j'k'}(\varphi_c, \theta_c) = \sum_{i=1}^{3}\sum_{j=1}^{3}\sum_{k=1}^{3} T_{i'i}(\varphi_c, \theta_c, 0) T_{j'j}(\varphi_c, \theta_c, 0) T_{k'k}(\varphi_c, \theta_c, 0)\, \chi^{(2)}_{ijk}, \tag{2.67}$$

Because a sphere is isotropic, no further orientational average is required; that is why in equation (2.62) no $< . >$ is needed. In other term, one can considered that $\Gamma^{(2)}_{i'j'k'}$ is equal to $\Gamma^{(2)}_{IJK}$. The $(a_V, b_V, c_V, a_H, b_H, c_H)$ coefficients and I_2, I_4 ratios as determined by:

$$a^{\Theta}_{V} = \Gamma_{XXX}\Gamma^{*}_{XXX} \tag{2.68}$$

$$c^{\Theta}_{V} = \Gamma_{XYY}\Gamma^{*}_{XYY} \tag{2.69}$$

$$b^{\Theta}_{V} = 4\Gamma_{XXY}\Gamma^{*}_{XXY} + \Gamma_{XXX}\Gamma^{*}_{XYY} + \Gamma^{*}_{XXX}\Gamma_{XYY} \tag{2.70}$$

$$a^{\Theta}_{H} = \Gamma_{YXX}\Gamma^{*}_{YXX}\cos^2\Theta + \Gamma_{ZXX}\Gamma^{*}_{ZXX}\sin^2\Theta - (\Gamma_{YXX}\Gamma^{*}_{ZXX} + \Gamma_{ZXX}\Gamma^{*}_{YXX})\cos\Theta\sin\Theta \tag{2.71}$$

$$c^{\Theta}_{H} = \Gamma_{YYY}\Gamma^{*}_{YYY}\cos^2\Theta + \Gamma_{ZYY}\Gamma^{*}_{ZYY}\sin^2\Theta - (\Gamma_{YYY}\Gamma^{*}_{ZYY} + \Gamma_{ZYY}\Gamma^{*}_{YYY})\cos\Theta\sin\Theta \tag{2.72}$$

$$\begin{aligned} b^{\Theta}_{H} = {} & (4\Gamma_{ZXY}\Gamma^{*}_{ZXY} + \Gamma_{ZXX}\Gamma^{*}_{ZYY} + \Gamma^{*}_{ZXX}\Gamma_{ZYY})\sin^2\Theta \\ & + (4\Gamma_{YYX}\Gamma^{*}_{YYX} + \Gamma_{YXX}\Gamma^{*}_{YYY} + \Gamma^{*}_{YXX}\Gamma_{YYY})\cos^2\Theta \\ & - \cos\Theta\sin\Theta(4\Gamma_{YYX}\Gamma^{*}_{ZXY} + 4\Gamma_{ZXY}\Gamma^{*}_{YYX} + \Gamma_{ZXX}\Gamma^{*}_{YYY} + \Gamma^{*}_{ZXX}\Gamma_{YYY} \\ & \quad + \Gamma_{YXX}\Gamma^{*}_{ZYY} + \Gamma_{ZYY}\Gamma^{*}_{YXX}) \end{aligned} \tag{2.73}$$

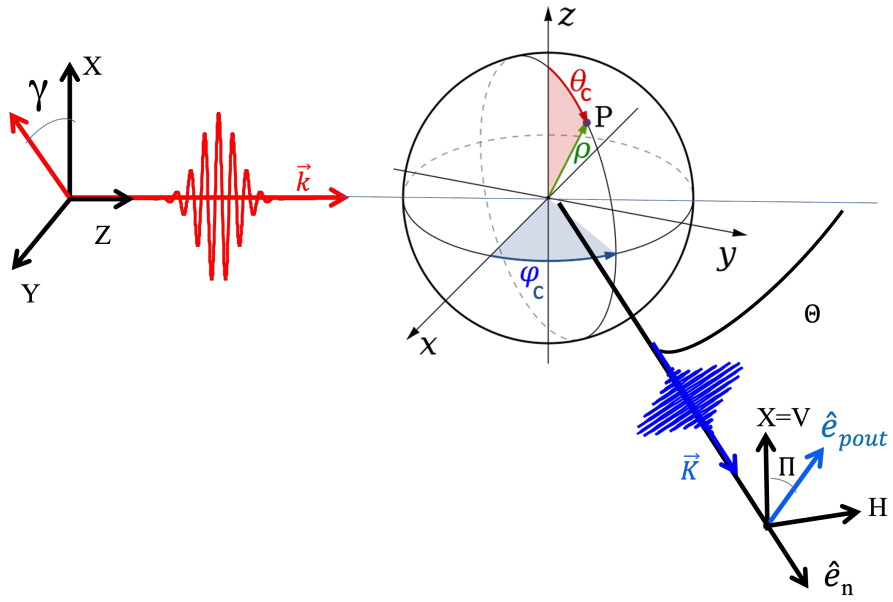


Figure 2.4: Definition of the angle (φ_c, θ_c) which describe the position on the surface sphere.

2.4.2 Computation with the Nonlinear Mie theory

The NLWKB/RGD models of the previous sections evaluate the surface response with an approximate propagation of the fundamental field. For a sphere an *exact* treatment is available: the linear Mie field at the fundamental frequency is used as the source of a nonlinear surface polarisation sheet, and the second-harmonic field is obtained by a multipole expansion (Mie-type) with the exact boundary conditions at 2ω. This is the model implemented in `shs_mie.py`; a detailed step-by-step derivation is given in the companion note `Documentation/shs_mie_theorie.tex`. The linear part follows Bohren and Huffman [4]; the surface-sheet source is of the type introduced for second-harmonic scattering by spheres in [5, 6].

Physical model

A dielectric sphere of radius a and (complex) index $n_s(\omega)$, $n_s(2\omega)$, surrounded by a medium of real index $n_m(\omega)$, $n_m(2\omega)$, is illuminated by a plane wave of frequency ω (propagation $+z$, polarisation x). Because bulk electric-dipole $\chi^{(2)}$ is forbidden in a centrosymmetric material, the nonlinear source is a *surface* polarisation sheet at $r = a$, built from the linear field $\mathbf{E}$ of the fundamental frequency at the interface:

$$p_r = \chi_{rrr}\, E_r^2 + \chi_{rtt}\big(E_\theta^2 + E_\varphi^2\big), \tag{2.74}$$

$$p_\theta = 2\,\chi_{trt}\, E_r\, E_\theta, \qquad p_\varphi = 2\,\chi_{trt}\, E_r\, E_\varphi. \tag{2.75}$$

The three constants describe an isotropic, achiral surface; in terms of the surface-frame tensor used by all the other modules (with z the normal),

$$\chi_{rrr} = \chi_{zzz}, \qquad \chi_{rtt} = \tfrac{1}{2}\big(\chi_{zxx} + \chi_{zyy}\big), \qquad \chi_{trt} = \tfrac{1}{4}\big(\chi_{xzx} + \chi_{xxz} + \chi_{yzy} + \chi_{yyz}\big). \tag{2.76}$$

All the other components of the tensor (chiral components, in-plane anisotropy) cannot be represented and are ignored; the program prints a warning that lists them. The radial field is discontinuous at the interface, so one must specify on which side $\mathbf{E}$ is evaluated: `side=outside` (default), `inside` or the mean of both sides.

Linear Mie field at ω

With the size parameters and relative indices

$$k_1 = \frac{2\pi n_m(\omega)}{\lambda}, \quad x_1 = k_1 a, \quad m_1 = \frac{n_s(\omega)}{n_m(\omega)}, \tag{2.77}$$

$$k_2 = \frac{2\pi n_m(2\omega)}{\lambda/2}, \quad x_2 = k_2 a, \quad m_2 = \frac{n_s(2\omega)}{n_m(2\omega)}, \tag{2.78}$$

the Bohren–Huffman coefficients a_n, b_n, c_n, d_n expand the incident, scattered and internal fields on the vector spherical harmonics $\mathbf{M}_{o1n}$, $\mathbf{N}_{e1n}$. The tangential electric field is continuous at $r = a$, whereas E_r is not (εE_r is). The number of multipoles is given by the Wiscombe criterion $n_{\max} = \lceil x + 4x^{1/3} + 2 \rceil$.

Multipole solution at 2ω

The sheet is projected on the scalar and vector spherical harmonics ($Y_{nm}, \mathbf{B}_{nm}, \mathbf{C}_{nm}$). For a linearly polarised excitation, the quadratic source only contains the azimuthal orders $m \in \{0, \pm 2\}$. The jump conditions at $r = a$,

$$\Delta \mathbf{E}_t = -\frac{1}{\varepsilon_s a} \sum_{nm} p^r_{nm}\, \mathbf{B}_{nm}, \tag{2.79}$$

$$\hat{\mathbf{r}} \times \Delta \tilde{\mathbf{H}} = -ik_0\, \mathbf{p}_t, \tag{2.80}$$

decouple into 2×2 TM (a_{nm}) and TE (b_{nm}) systems whose outgoing solutions are

$$a^{\text{out}}_{nm} = \frac{n_s\, j\, p^r_{nm}/(\varepsilon_s a) + k_0 \alpha_{nm} D_j}{n_m h D_j - n_s j D_h}, \tag{2.81}$$

$$b^{\text{out}}_{nm} = \frac{-k_0 \beta_{nm}\, j}{n_m j D_h - n_s h D_j}, \tag{2.82}$$

where j, h are the spherical Bessel and Hankel functions of order n at x_2, D_j, D_h the associated Riccati–Bessel derivative terms, and α_{nm}, β_{nm} the projections of the tangential sheet on $\mathbf{B}_{nm}$ and $\mathbf{C}_{nm}$.

Far field, polarisation and units

In the radiation zone $\mathbf{E}_{\text{SH}} \simeq e^{ik_2 r}\, \mathbf{F}(\theta, \varphi)/(k_2 r)$, with $\mathbf{F}$ built from a^{out}_{nm}, b^{out}_{nm} and the angular vector harmonics, and the intensities are

$$I_\theta = \frac{|F_\theta|^2}{k_2^2}, \qquad I_\varphi = \frac{|F_\varphi|^2}{k_2^2}, \tag{2.83}$$

that is $r^2 |\mathbf{E}_{2\omega}|^2$ for $E_0 = 1$: they are in the absolute units `rad` of Section 2.8.4 (radiative prefactor included). The channels HH, HV, VH, VV (first letter: incident polarisation; second: detected polarisation) follow from the azimuth of the detection plane, and the curves versus the incident polarisation angle γ use $\varphi_d = \pi/2 - \gamma$. The scattering-angle convention is that of the other modules ($\Theta = 0$: forward direction).

Relation with the NLWKB/RGD model and numerical checks

- When there is no index contrast ($n_s = n_m$), the fundamental field is the incident plane wave, E_r is continuous and the Mie result must coincide with the NLRGD model. This is verified numerically to 2×10^{-7}, *in absolute units* (Figure 2.5a).

- With an index contrast the two models differ (Figure 2.5b,c: the Mie intensity is 1.6–1.7 times the NLWKB one near the maxima for $n_p = 1.5$ in $n_m = 1.33$, and the minima are shifted): the NLWKB model is an approximation of the propagation of the fundamental field, whereas the Mie theory is exact for the sphere.
- For an achiral sphere with linear incident polarisation, $I_{HV} = I_{VV} = 0$ by symmetry.

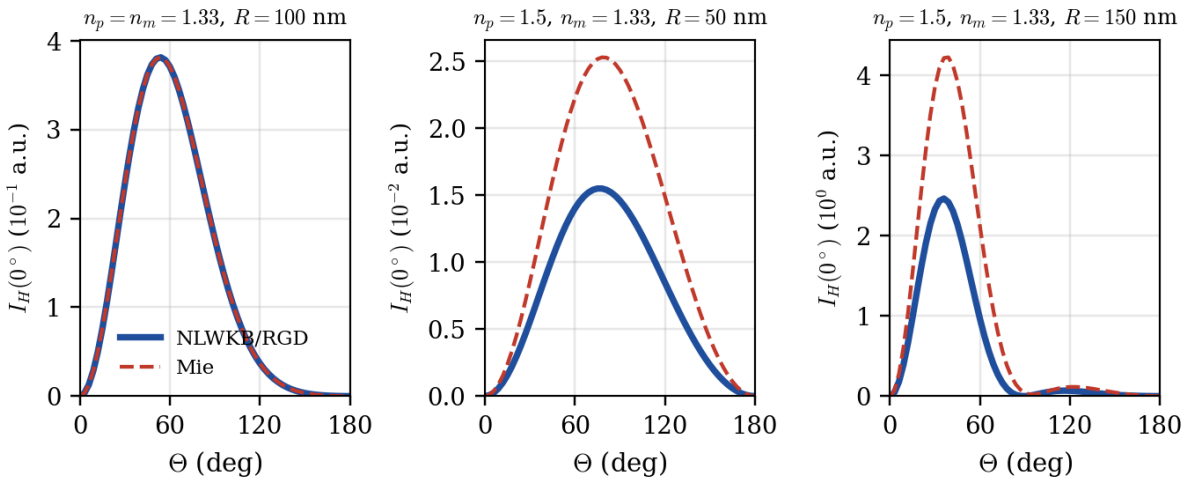


Figure 2.5: Nonlinear Mie theory (`shs_mie.py`, dashed) compared with the NLWKB/RGD model (`sphere_SHS.py`, solid), $I_H(0°)$ versus the scattering angle, $\lambda = 800$ nm, $\chi_{zzz} = 1$, $\chi_{zxx} = \chi_{zyy} = 0.3$, $\chi_{xxz} = \chi_{xzx} = \chi_{yyz} = \chi_{yzy} = 0.2$. (a) Index-matched sphere ($R = 100$ nm): the two curves coincide. (b), (c) $n_p = 1.5$ in $n_m = 1.33$, $R = 50$ and 150 nm: the models differ.

2.5 Coherent SHS by a cylinder

2.5.1 Physical model

As was the case for the sphere module, for the cylinder, the surrounding medium and the particle interior are optically distinct. Two complex refractive indices enter the calculation:

- n_p: refractive index of the *particle* (cylinder material),
- n_m: refractive index of the surrounding *medium*.

The wave-vector magnitudes are

$$k_\omega = \frac{2\pi n_p}{\lambda} \qquad \text{(fundamental, inside the particle)}, \tag{2.84}$$

$$k_{2\omega} = \frac{4\pi n_m}{\lambda} \qquad \text{(harmonic, in the medium)}. \tag{2.85}$$

2.5.2 Geometry and local surface frame

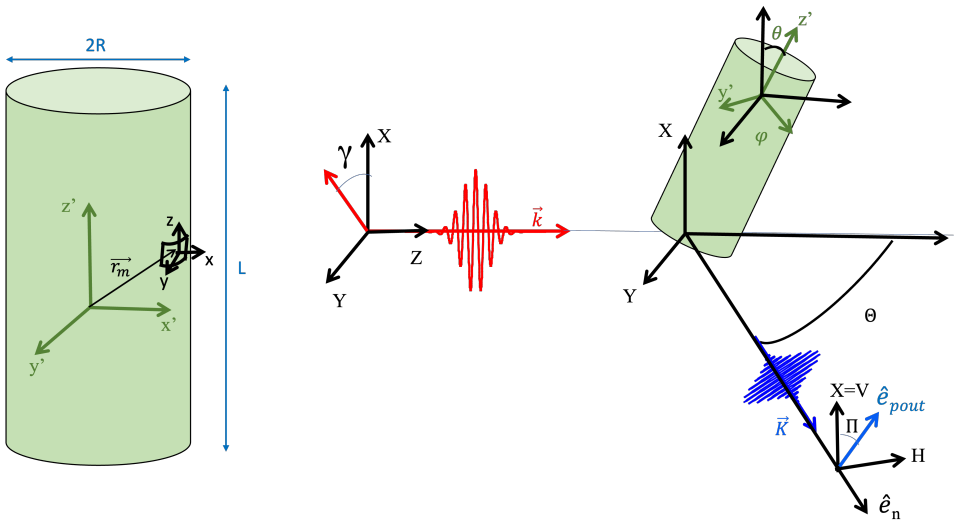


Figure 2.6: Cylinder of radius R and length L. A surface element is labeled by the azimuthal angle $\theta_c \in [0, 2\pi]$ and the axial coordinate $z_c \in [-L/2, L/2]$. The local surface frame is labeled by (x,y,z) , the cylinder frame by (x',y',z') and it is oriented in the laboratory frame (X,Y,Z) by Euler angles (ϕ, θ, ψ).

A point on the lateral surface has mesoscopic Cartesian coordinates

$$x' = R\cos\theta_c, \qquad y' = R\sin\theta_c, \qquad z' = z_c. \tag{2.86}$$

Its laboratory coordinates are obtained with the ZYZ rotation matrix $\mathbf{T}(\varphi, \theta, \psi)$:

$$r_i^{\text{lab}}(\varphi,\theta,\psi) = T_{i0}(\varphi,\theta,\psi)\, x' + T_{i1}(\varphi,\theta,\psi)\, y' + T_{i2}(\varphi,\theta,\psi)\, z', \qquad i \in \{x, y, z\}. \tag{2.87}$$

2.5.3 Effective nonlinear susceptibility

The scattered intensity is computed from an effective susceptibility that integrates the local response over the whole lateral surface [1]:

$$I_{SHS}(\mathbf{e}^{\omega}_{p-in}, \mathbf{e}^{2\omega}_{p-out}, \mathbf{e}^{2\omega}_{n}) = I_{SHS}(\gamma, \Pi, \Theta) =< \mathbf{\Gamma}_{eff}(\gamma, \Pi, \Theta).\mathbf{\Gamma}^{*}_{eff}(\gamma, \Pi, \Theta) > \tag{2.88}$$

Note that contrary to the sphere, the average over all cylinder orientation $< . >$ is here necessary as the cylinder is not isotropic. $\mathbf{\Gamma}_{eff}$ is defined by:

$$\mathbf{\Gamma}_{eff} = \mathbf{e}^{2\omega}_{n} \times (\overleftrightarrow{\mathbf{\Gamma}^{(2)}} : \mathbf{e}^{\omega}_{p-in}\mathbf{e}^{\omega}_{p-in}) \times \mathbf{e}^{2\omega}_{n} \tag{2.89}$$

and

$$\overleftrightarrow{\mathbf{\Gamma}^{(2)}} = \iint_{surface} \overleftrightarrow{\chi^{(2)}} e^{i\Delta\mathbf{k}.\mathbf{r}} dS \tag{2.90}$$

Each Cartesian component of $\overleftrightarrow{\mathbf{\Gamma}^{(2)}}$ is

$$\overleftrightarrow{\Gamma}^{(2)}_{IJK}(\varphi,\theta,\psi,\Theta) = \int_0^{2\pi} \mathrm{d}\theta_c \int_{-L/2}^{L/2} \mathrm{d}z_c\; R\, \overleftrightarrow{\chi}^{(2)}_{IJK}(\theta_c, z_c, \varphi, \theta, \psi)\, e^{\,i\,\Phi(\theta_c, z_c, \Theta, k^{\omega}, K^{2\omega})}, \tag{2.91}$$

where the NLWKB phase collected along the optical path is

$$\Phi(\theta_c, z_c, \Theta, k^{\omega}, K^{2\omega}) = \left(-2k^{\omega} + K^{2\omega}\cos\Theta\right) r_z^{\text{lab}} + K^{2\omega}\sin\Theta\, r_y^{\text{lab}}. \tag{2.92}$$

The laboratory components r_y^{lab} and r_z^{lab} are those of (2.87).

The $\overset{\leftrightarrow(2)}{\chi}_{IJK}$ tensor elements expressed in the laboratory frame are deduced according to:

$$\overset{\leftrightarrow(2)}{\chi}_{IJK}(\theta_c, z_c, \varphi, \theta, \psi) = \sum_{i=1'}^{3}\sum_{j=1'}^{3}\sum_{k'=1}^{3} T_{Ii'}(\varphi,\theta,\psi)T_{Jj'}(\varphi,\theta,\psi)T_{Kk'}(\varphi,\theta,\psi)\,\overset{\leftrightarrow(2)}{\chi}_{i'j'k'}(\theta_c, z_c) \tag{2.93}$$

The i second-order susceptibility tensor $\chi^{(2)}_{i'j'k'}$ expressed in the cylinder frame is determined by:

$$\overset{\leftrightarrow(2)}{\chi}_{i'j'k'}(\theta_c) = \sum_{i=1}^{3}\sum_{j=1}^{3}\sum_{k=1}^{3} T_{i'i}(\theta_c, \frac{\pi}{2}, 0)T_{j'j}(\theta_c, \frac{\pi}{2}, 0)T_{k'k}(\theta_c, \frac{\pi}{2}, 0)\,\overset{\leftrightarrow(2)}{\chi}_{ijk} \tag{2.94}$$

where $\overset{\leftrightarrow(2)}{\chi}_{ijk}$ is the second order susceptibility tensor expressed in the local surface frame.

2.5.4 Numerical quadrature and orientational average

The surface susceptibility $\chi^{(2)}_{i'j'k'}$ does not depend on z_c, so the integral along the axis in equation (2.91) is performed *analytically*: the phase is linear in z_c, $\Phi = \Phi_0(\theta_c) + \kappa\, z_c$ with $\kappa = (-2k^{\omega} + K^{2\omega}\cos\Theta)\,T_{Z3} + K^{2\omega}\sin\Theta\,T_{Y3}$ (T_{I3}: laboratory component I of the cylinder axis), and

$$\int_{-L/2}^{L/2} e^{\,i\kappa z_c}\, dz_c = L\ \mathrm{sinc}\Big(\frac{\kappa L}{2}\Big). \tag{2.95}$$

There is therefore no axial grid (and no aliasing for long rods). The integral around the axis is evaluated with a periodic trapezoid rule of n_β points, with $n_\beta \gtrsim q_{\max}R + 14$. Only the lateral surface is integrated: the two end caps are not modelled, so the model is meant for objects long compared with their radius. For an isotropic suspension of cylinders an additional average over the orientation of the cylinder axis (φ, θ) is required, with n_φ trapezoid points and n_θ Gauss–Legendre points in $\cos\theta$ ($n_\varphi = n_\theta \gtrsim q_{\max}\sqrt{R^2 + L^2/4} + 12$, Section 2.8.3):

$$\langle \Gamma^{(2)}_{IJK}(\Theta)\rangle = \frac{1}{4\pi}\int_0^{2\pi} \mathrm{d}\varphi \int_0^{\pi} \mathrm{d}\theta\, \sin\theta\, \Gamma^{(2)}_{IJK}(\varphi, \theta, \psi = 0, \Theta). \tag{2.96}$$

2.5.5 Polarization-resolved coefficients

From the averaged laboratory components the six coefficients can be deduced :

$$a_V^{\Theta} = < \Gamma_{XXX}\Gamma^*_{XXX} > \tag{2.97}$$

$$c_V^{\Theta} = < \Gamma_{XYY}\Gamma^*_{XYY} > \tag{2.98}$$

$$b_V^{\Theta} = < 4\Gamma_{XXY}\Gamma^*_{XXY} + \Gamma_{XXX}\Gamma^*_{XYY} + \Gamma^*_{XXX}\Gamma_{XYY} > \tag{2.99}$$

$$a_H^{\Theta} = < \Gamma_{YXX}\Gamma^*_{YXX} > \cos^2\Theta + < \Gamma_{ZXX}\Gamma^*_{ZXX} > \sin^2\Theta - < \Gamma_{YXX}\Gamma^*_{ZXX} + \Gamma_{ZXX}\Gamma^*_{YXX} > \cos\Theta\sin\Theta \tag{2.100}$$

$$c_H^{\Theta} = < \Gamma_{YYY}\Gamma^*_{YYY} > \cos^2\Theta + < \Gamma_{ZYY}\Gamma^*_{ZYY} > \sin^2\Theta - < \Gamma_{YYY}\Gamma^*_{ZYY} + \Gamma_{ZYY}\Gamma^*_{YYY} > \cos\Theta\sin\Theta \tag{2.101}$$

$$\begin{aligned} b_H^{\Theta} = {} & < 4\Gamma_{ZXY}\Gamma^*_{ZXY} + \Gamma_{ZXX}\Gamma^*_{ZYY} + \Gamma^*_{ZXX}\Gamma_{ZYY} > \sin^2\Theta \\ & + < 4\Gamma_{YYX}\Gamma^*_{YYX} + \Gamma_{YXX}\Gamma^*_{YYY} + \Gamma^*_{YXX}\Gamma_{YYY} > \cos^2\Theta \\ & - \cos\Theta\sin\Theta < 4\Gamma_{YYX}\Gamma^*_{ZXY} + 4\Gamma_{ZXY}\Gamma^*_{YYX} + \Gamma_{ZXX}\Gamma^*_{YYY} + \Gamma^*_{ZXX}\Gamma_{YYY} \\ & \qquad + \Gamma_{YXX}\Gamma^*_{ZYY} + \Gamma_{ZYY}\Gamma^*_{YXX} > \end{aligned} \tag{2.102}$$

The polarization-resolved intensities and the ratios $I_2^{\Theta,\Pi}$, $I_4^{\Theta,\Pi}$ also follow equations (2.20) to (2.25).

2.6 Coherent SHS by an ellipsoid of revolution

2.6.1 Physical model

The ellipsoid module treats a spheroid (ellipsoid of revolution) in the NLWKB/RGD framework of Section 2.5: the surface response is integrated over the whole surface, the particle and the medium have refractive indices n_p and n_m, and the wave-vector magnitudes are those of equations (2.84)–(2.85). The classical Rayleigh–Gans–Debye limit is obtained with $n_p = n_m$. The particle frame (x', y', z') is attached to the symmetry axis z' and the particle is oriented in the laboratory frame by the ZYZ Euler angles (φ, θ, ψ) of Section 2.5. Because the particle is a body of revolution, the angle ψ is irrelevant and is set to 0.

2.6.2 Geometry and local surface frame

The surface is parametrised by the polar angle $\alpha \in [0, \pi]$ and the azimuthal angle $\beta \in [0, 2\pi[$ (Figure 2.7):

$$x' = a \sin\alpha \cos\beta, \qquad y' = a \sin\alpha \sin\beta, \qquad z' = c \cos\alpha, \tag{2.103}$$

where a is the equatorial radius and $c = L/2$ the half-length along the symmetry axis ($c > a$: prolate, $c < a$: oblate, $c = a$: sphere). The surface element and the outward unit normal are

$$dS = a \sin\alpha \, N(\alpha) \, d\alpha \, d\beta, \qquad N(\alpha) = \sqrt{c^2 \sin^2\alpha + a^2 \cos^2\alpha}, \qquad \mathbf{n} = \frac{1}{N} \begin{pmatrix} c \sin\alpha \cos\beta \\ c \sin\alpha \sin\beta \\ a \cos\alpha \end{pmatrix}. \tag{2.104}$$

The local surface frame (x, y, z) in which the surface susceptibility $\overset{\leftrightarrow}{\chi}{}^{(2)}_{ijk}$ is given is defined at every point by

$$\begin{aligned} \mathbf{e}_z &= \mathbf{n}, \\ \mathbf{e}_x &= \mathbf{t} = \frac{1}{N} \begin{pmatrix} a \cos\alpha \cos\beta \\ a \cos\alpha \sin\beta \\ -c \sin\alpha \end{pmatrix} \quad \text{(meridian tangent)}, \\ \mathbf{e}_y &= \begin{pmatrix} -\sin\beta \\ \cos\beta \\ 0 \end{pmatrix} \quad \text{(azimuthal tangent)}. \end{aligned} \tag{2.105}$$

This frame is right-handed ($\mathbf{e}_x \times \mathbf{e}_y = \mathbf{e}_z$) and it reduces, for $c = a$, to the frame $T(\varphi_c, \theta_c, 0)$ used for the sphere; for a cylinder-like surface point on the equator it coincides with the frame of Section 2.5. In terms of Euler angles it is $T(\beta, \theta_n, 0)$ with θ_n the polar angle of the normal, $\tan\theta_n = (c/a)\tan\alpha$; this is how the dipole assemblies of `script_spheroid_random.py` are oriented (Chapter 10).

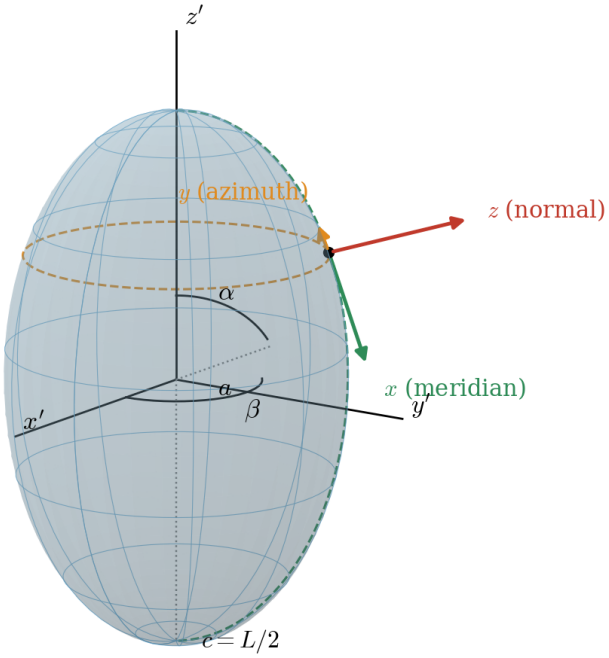


Figure 2.7: Ellipsoid of revolution of equatorial radius a and half-length $c = L/2$. A surface point is labelled by (α, β); the local surface frame is x (meridian tangent), y (azimuthal tangent) and z (outward normal). (x', y', z') is the particle frame.

The surface susceptibility in the particle frame is obtained as for the sphere and the cylinder,

$$\chi^{(2)}_{i'j'k'}(\alpha,\beta) = \sum_{i,j,k=1}^{3} S_{i'i}\, S_{j'j}\, S_{k'k}\, \chi^{(2)}_{ijk}, \qquad S = \left[\, \mathbf{e}_x \;\; \mathbf{e}_y \;\; \mathbf{e}_z \,\right] \text{ (columns)}, \tag{2.106}$$

and in the laboratory frame by $\chi^{(2)}_{IJK} = \sum T_{Ii'} T_{Jj'} T_{Kk'} \chi^{(2)}_{i'j'k'}$ as in equation (2.93).

2.6.3 Effective susceptibility and orientational average

The intensity is given by equations (2.88) and the effective tensor by the surface integral

$$\Gamma^{(2)}_{IJK}(\varphi,\theta,\Theta) = \int_0^{2\pi} d\beta \int_0^{\pi} d\alpha \; a \sin\alpha \, N(\alpha) \; \chi^{(2)}_{IJK}(\alpha,\beta;\varphi,\theta) \; e^{\,i\,\Phi(\alpha,\beta,\Theta)}, \tag{2.107}$$

where Φ is the phase (2.92) evaluated with the laboratory coordinates $\mathbf{r}^{\mathrm{lab}} = T\,\mathbf{r}'$ of the surface point (2.103). With the variable $u = \cos\alpha$ the area element is $a\,N(u)\,du\,d\beta$ with $N(u) = \sqrt{c^2(1-u^2) + a^2u^2}$, a smooth function of u; this is why the α integral is evaluated by a Gauss–Legendre rule in u (Section 2.8.2). The particle is not isotropic (except for $c = a$), so the average over the orientation of the symmetry axis is required,

$$\langle\, \cdot \,\rangle = \frac{1}{4\pi} \int_0^{2\pi} d\varphi \int_0^{\pi} \sin\theta \, d\theta \; (\,\cdot\,)_{\psi=0}, \tag{2.108}$$

and the six coefficients $a^{\Theta}_V, \ldots, c^{\Theta}_H$ are the orientational averages of the same products of Γ components as in Section 2.5 (the equations of that section apply unchanged), followed by the polarisation-resolved intensities (2.12)–(2.13). For $c = a$ the module reproduces the sphere for any surface tensor.

2.7 Coherent SHS by a rectangular box (cube)

2.7.1 Physical model and geometry

The box module computes the SHS of a rectangular parallelepiped of edges a, b, c along x', y', z' (a cube for $a = b = c$) whose *six faces* carry the surface susceptibility, with the same NLWKB/RGD wave vectors as in the previous sections. The faces have the outward normals $\mathbf{n} = \pm\mathbf{e}_{x'}, \pm\mathbf{e}_{y'}, \pm\mathbf{e}_{z'}$ (Figure 2.8). Each face is the rectangle

$$\mathbf{r}(s,t) = \mathbf{r}_0 + \left(s - \tfrac{1}{2}\right)\mathbf{u} + \left(t - \tfrac{1}{2}\right)\mathbf{v}, \qquad s,t \in [0,1], \tag{2.109}$$

with $\mathbf{r}_0$ the centre of the face, $\mathbf{u}$, $\mathbf{v}$ the two full edge vectors and area $A_f = |\mathbf{u}||\mathbf{v}|$.

The local frame $(\mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z)$ of each face is right-handed with $\mathbf{e}_z = \mathbf{n}$:

$$\text{lateral faces } (\mathbf{n} \perp \mathbf{e}_{z'}): \quad \mathbf{e}_x = -\mathbf{e}_{z'}, \ \ \mathbf{e}_y = \mathbf{e}_{z'} \times \mathbf{n}; \qquad \text{caps } (\mathbf{n} = \pm\mathbf{e}_{z'}): \quad \mathbf{e}_x = n_{z'}\,\mathbf{e}_{x'}, \ \ \mathbf{e}_y = \mathbf{e}_{y'}. \tag{2.110}$$

Since the frame is constant on a face, $\chi^{(2)}_{i'j'k'}$ is also constant on it and is obtained from equation (2.106) with the corresponding matrix S_f.

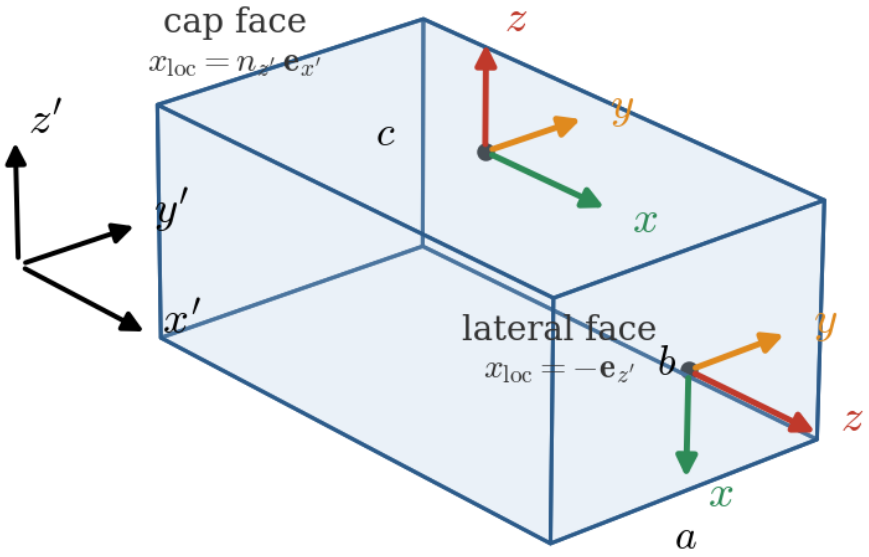


Figure 2.8: Rectangular box of edges $a \times b \times c$ with the particle frame (x', y', z'). The local surface frame (x, y, z) of a lateral face and of a cap face is drawn (z = outward normal); the other faces are obtained by symmetry.

2.7.2 Effective susceptibility

Because $\chi^{(2)}$ is uniform on a face, the surface integral of each face is exact (no quadrature): with Φ_c the phase (2.92) at the centre of the face and $\Phi_u = \Delta\mathbf{k}\cdot\mathbf{u}$, $\Phi_v = \Delta\mathbf{k}\cdot\mathbf{v}$ its variations along the two edges (laboratory components, $\Delta\mathbf{k}$ as in equation (2.92)),

$$\Gamma^{(2),f}_{IJK} = A_f\,\chi^{(2),f}_{IJK}\,e^{i\Phi_c}\,\operatorname{sinc}\!\left(\frac{\Phi_u}{2}\right)\operatorname{sinc}\!\left(\frac{\Phi_v}{2}\right), \qquad \operatorname{sinc} x = \frac{\sin x}{x}, \qquad \Gamma^{(2)}_{IJK} = \sum_{f=1}^{6}\Gamma^{(2),f}_{IJK}. \tag{2.111}$$

The coherent sum over the six faces gives the interference pattern of the box; the sinc factors give the diffraction by each face. The box has no continuous symmetry: the average over its orientation uses all three Euler angles,

$$\langle\,\cdot\,\rangle = \frac{1}{8\pi^2}\int_0^{2\pi} d\varphi \int_0^{\pi} \sin\theta\, d\theta \int_0^{2\pi} d\psi\ (\,\cdot\,), \tag{2.112}$$

exactly as for the molecular assembly of equation (2.28), and the coefficients $a_V^\Theta, \ldots, c_H^\Theta$ are those of Section 2.5.

2.8 Numerical conventions common to all modules

This section collects the conventions shared by the programs; the chapters of the programs refer to it.

2.8.1 Symmetry of the tensor

The polarisation expressions of Sections 2.3 and 2.5 use only the components $(j,k) = (xx), (yy), (xy)$ of the last two indices and assume the intrinsic permutation symmetry $\beta_{ijk} = \beta_{ikj}$ (the factor 2 of the cross terms). Every module therefore **symmetrises its input tensor**,

$$\beta_{ijk} \;\leftarrow\; \tfrac{1}{2}\big(\beta_{ijk} + \beta_{ikj}\big), \tag{2.113}$$

when reading the tensor file, and prints a warning if the file was not symmetric. A tensor that is not symmetric would otherwise give module-dependent results. (This is not the Kleinman symmetry, which would also permute the first index.)

2.8.2 Orientation quadrature

All orientational averages (2.28), (2.96), (2.108), (2.112) are evaluated with a *regular* grid in the azimuthal angles φ and ψ (periodic trapezoid rule, exact for harmonics of order lower than the number of points) and a **Gauss–Legendre rule in** $\cos\theta$ with the weight $\sin\theta\, d\theta = d(\cos\theta)$. The weights then sum *exactly* to $8\pi^2$ (or 4π for two angles), so the normalisation $1/(8\pi^2)$ is exact. The former mid-point rule in θ had a weight sum of $1.0041 \times 8\pi^2$, i.e. a +0.4% bias on rotation-invariant quantities (`PYSHS_THETA_RULE=midpoint` restores it). The surface integrals of the sphere and of the ellipsoid use the same Gauss–Legendre rule in the cosine of the polar angle of the surface.

2.8.3 Automatic size of the grids

All the integrals contain the phase factor $e^{i\,\mathbf{q}\cdot\mathbf{r}}$ with $|\mathbf{q}| \le q_{\max} = 2\,\mathrm{Re}\,k^\omega + \mathrm{Re}\,K^{2\omega}$. A grid that is fine enough for a small object silently becomes too coarse for a large one, so the grids are sized from the dimensionless *size parameter* $q_{\max}\ell$, with ℓ the largest half-extent of the object. The rules were calibrated against much finer reference grids (relative error $\lesssim 10^{-4}$, usually $\sim 10^{-7}$ for continuum objects) and are summarised in Table 2.1; the historical sizes are always used as minimum values, and the grid actually used is printed in the log of each run.

Module	Quantity	Rule (minimum: historical size)
`sphere_SHS`	n_y points in $\cos\theta_c$ (Gauss), $n_x = 2n_y$ in φ_c	$n_y \ge q_{\max}R/2 + 20$ (≥ 40)
`cylinder_SHS`	azimuth n_β; orientation $n_\varphi = n_\theta$	$n_\beta \ge q_{\max}R + 14$; $n_\varphi \ge q_{\max}\sqrt{R^2 + L^2/4} + 12$
	axial integral	analytic (sinc), no grid
`ellipsoid_SHS`	surface $n_\alpha \times n_\beta$; orientation $n_\varphi = n_\theta$	$n_\alpha \ge q_{\max}\ell/2{+}20$, $n_\beta \ge q_{\max}\ell{+}16$, $n \ge q_{\max}\ell{+}12$; $\ell = \max(a,c)$
`cube_SHS`	orientation $N \times N \times N$	$N \ge q_{\max}d + 12$ (≥ 24), $d = \frac{1}{2}\sqrt{a^2+b^2+c^2}$
	faces	analytic (sinc), no grid
`SHS`	orientation $N_\varphi \times N_\theta \times N_\psi$	$N_\varphi = N_\psi \ge 2q_{\max}r_{\max} + 8$ (≥ 20), $N_\theta = N_\varphi/2$ (≥ 10)
`HRS`	orientation $20 \times 10 \times 20$	none (single molecule, no phase)
`shs_mie`	multipoles, quadrature in (θ, φ)	Wiscombe criterion (already adaptive)

Table 2.1: Automatic grid rules; $q_{\max} = 2\,\mathrm{Re}\,k^\omega + \mathrm{Re}\,K^{2\omega}$, $r_{\max}$ is the largest distance of a dipole to the origin.

Upper bounds protect the memory and the run time (for instance $N \leq 80$ for the box and for SHS, the cost growing as N^3); when a bound prevents the rule from being satisfied a `WARNING` is printed. The environment variables `PYSHS_GRID=fixed` (historical fixed grids), `PYSHS_GRID_SCALE=`s (multiplies the size parameter: $s > 1$ safer and slower) and `PYSHS_GRID_MAX=`n (upper bound) control the behaviour.

2.8.4 Intensity units

Two conventions are used, related by the radiative prefactor $P = (|K^{2\omega}|^2/4\pi)^2$ ($K^{2\omega} = 4\pi n_m/\lambda$ in nm^{-1}):

`rad` $I = r^2|\mathbf{E}_{2\omega}|^2/|\mathbf{E}_{\omega}|^4$, **with** the prefactor. It is the default of the sphere, cylinder, ellipsoid, box and Mie modules.

`raw` orientational average of $\left|\sum \beta\, e^{i\mathbf{q}\cdot\mathbf{r}}\right|^2$, **without** the prefactor, with β in atomic units (for instance $\langle\beta^2_{ZZZ}\rangle = 1/7$ for $\beta_{zzz} = 1$). It is the default of HRS and SHS.

$$I_{\mathrm{rad}} = P\, I_{\mathrm{raw}}. \tag{2.114}$$

`PYSHS_UNITS=rad` or `raw` forces one convention on every module and `PYSHS_I_SCALE` multiplies every result (the GUI has an *Intensity units* selector, Chapter 9); the convention used is printed in the log and in the header of the output file. A continuum model with surface susceptibility χ is *equivalent* to a dipole assembly of surface density σ (dipoles per nm^2) and molecular hyperpolarizability β with

$$\chi = \sigma\,\beta \qquad \Longrightarrow \qquad I_{\mathrm{SHS}}(\text{assembly}) = \sigma^2\, I_{\text{continuum}} \quad \text{(same units)}, \tag{2.115}$$

so χ is expressed in "units of β per nm^2".

2.8.5 Scattering-angle grid

In the `angle_scattering` mode the number N requested is the number of intervals between $0°$ and $180°$, and $N+1$ angles $\Theta = 0, 180/N, \ldots, 180°$ are computed in *every* module.

2.8.6 Absorbing media and origin

With complex refractive indices the phase $e^{i\mathbf{q}\cdot\mathbf{r}}$ contains the attenuation $e^{-\mathrm{Im}\,\mathbf{q}\cdot\mathbf{r}}$, which depends on the origin of the positions: place the origin where the incident beam enters the object. For real indices a global translation only changes the phase and has no effect on the intensity.

2.8.7 Chiral cross terms and the legacy mode

The odd terms $b_1 c^3 s$ and $b_2 c s^3$ ($c = \cos\gamma$, $s = \sin\gamma$) of the exact expansion of $I(\gamma)$ are not written in equations (2.12)–(2.13) because they **vanish identically after an isotropic orientational average**: $b_1 \propto \mathrm{Re}\langle A_{xx} A^*_{xy}\rangle$ is the real part of a Hermitian-symmetric contraction, to which the chiral invariants (ε_{ijk}) contribute only through the imaginary part (circular-polarisation effects). Numerically $|b_1|, |b_2| \lesssim 10^{-16}\max(a,b,c)$ even for chiral assemblies. The `--legacy` option of HRS and SHS reproduces two defects of the former C code (the missing $8\pi^2$ normalisation of b_1, b_2 and a wrong pairing of the cross terms of b_{H1}, b_{H2}): it therefore only changes rounding noise in these four coefficients and none of the reported quantities (a, b, c, I_2, I_4).

Chapter 3

HRS program

The `HRS.py` program implements the incoherent scattering of Section 2.2: a single molecule, orientationally averaged, without any phase factor.

3.1 Inputs and outputs

The only molecular input is the hyperpolarizability tensor β_{ijk} in atomic units, provided in an `input_beta` file:

```
# comment line
xxx  0.0
xxy  0.0
xxz  0.0
...
zzz  value
```

(27 components, one per line after the header, label then value). The tensor is symmetrised in its last two indices when it is read (Section 2.8.1).

Interactive parameters (stdin): wavelength λ (nm), real and imaginary parts of the refractive index n, then scattering angle(s) according to the keyword. The wavelength and the index are read to keep the same input protocol as the other modules; they do not change the default `raw` intensities and only enter the radiative prefactor when `PYSHS_UNITS=rad` is requested (Section 2.8.4).

Outputs: coefficients $a_V, b_V, c_V, b_{V1}, b_{V2}, a_H, b_H, c_H, b_{H1}, b_{H2}$, ratios $I_{2V}, I_{4V}, I_{2H}, I_{4H}$, and the file `out_plot` for polarization or angular curves.

3.2 Command line

The program is the console command `pyshs-optics-hrs` installed by `pip`, also available as `python3 -m py_shs.HRS` (Section 1.4). The same rule holds for the other programs of this manual.

```
pyshs-optics-hrs <keyword> <input_beta> <output_file> [--legacy]
```

The keyword is one of:

`polarplot_single` one scattering angle Θ; the curves $I_V(\gamma)$ and $I_H(\gamma)$ for $\gamma \in [0, 2\pi[$ (100 points) are written in `out_plot`;

`polarplot_integrate` the same curves averaged over a range of scattering angles (plain average of the intensities over the points, Section 2.8);

`angle_scattering` the four intensities $I_H(0°)$, $I_H(90°)$, $I_V(0°)$, $I_V(90°)$ as functions of Θ ($N+1$ angles between 0° and 180°, Section 2.8.5).

Example ($\beta_{zzz} = 1$, all other components zero, $\lambda = 800$ nm, $n = 1.33$, $\Theta = 90°$):

```
printf "800\n1.33\n0.0\n90\n" | \
  pyshs-optics-hrs polarplot_single beta.txt result.txt
```

Optional flag `--legacy` (or `PYSHS_LEGACY=1`) reproduces two defects of the historical C code; it does not change any reported quantity (Section 2.8.7).

3.3 Example

For the one-dimensional molecule $\beta_{zzz} = 1$ the orientational averages are known analytically, which makes this case a useful check of the installation. With $\mathbf{e}_V = \mathbf{e}_X$ one finds at $\Theta = 90°$

$$a_V = \tfrac{1}{7}, \quad b_V = \tfrac{6}{35}, \quad c_V = \tfrac{1}{35}, \qquad a_H = c_H = \tfrac{1}{35}, \quad b_H = \tfrac{2}{35}, \tag{3.1}$$

hence $D = c_V/a_V = 1/5$ and $I_{2V} = 2/3$, which are the values printed by the program:

```
**********calculation complete ************
av = 0.142857
bv = 0.171429
bv1 = 0.000000
bv2 = 0.000000
cv = 0.028571
ah = 0.028571
bh = 0.057143
bh1 = 0.000000
bh2 = -0.000000
ch = 0.028571
*******************************************
I2v = 0.666667
I4v = -0.000000
I2h = -0.000000
I4h = -0.000000
```

Figure 3.1 shows the polarisation curves and the angular dependence for this molecule. At $\Theta = 90°$, $I_V(\gamma) = a_V \cos^4\gamma + b_V \cos^2\gamma \sin^2\gamma + c_V \sin^4\gamma$ oscillates with a period of 180° between $a_V = 1/7$ (incident polarisation along X, $\gamma = 0°$) and $c_V = 1/35$ ($\gamma = 90°$), whereas I_H does not depend on γ: $I_H = \frac{1}{35}(\cos^2\gamma + \sin^2\gamma)^2 = 1/35$. The signal I_V does not depend on Θ (the detection direction $\mathbf{e}_V = \mathbf{e}_X$ is perpendicular to the scattering plane), while $I_H(90°)$ goes from a_V at $\Theta = 0$ to c_V at $\Theta = 90°$.

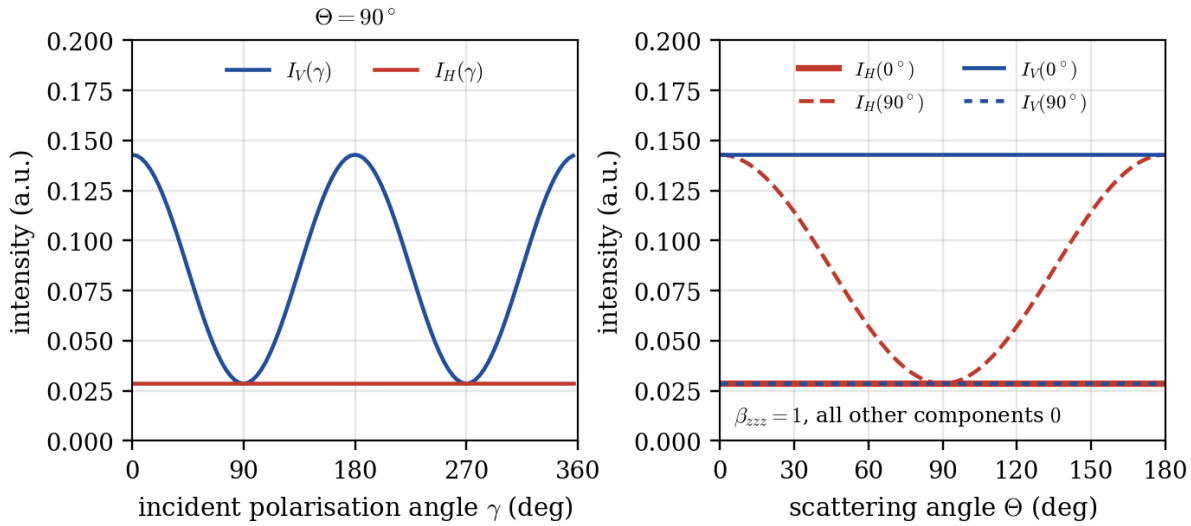


Figure 3.1: HRS of a molecule with $\beta_{zzz} = 1$ (default tensor of the GUI). Left: polarisation curves $I_V(\gamma)$, $I_H(\gamma)$ at $\Theta = 90°$ (`polarplot_single`). Right: $I_H(0°)$, $I_H(90°)$, $I_V(0°)$, $I_V(90°)$ versus the scattering angle (`angle_scattering`, $N = 90$). Atomic units, `raw` convention.

3.4 Code architecture

`stdin` (interactive or pipe)
↓
read λ, $n = n_r + in_i$, angles according to the mode
↓
read β and symmetrise it (`read_beta`)
↓
build the orientation grid (`Orientations`): $T(\varphi, \theta, \psi)$ and weights w
↓
rotate β to the laboratory for every orientation, *once*
(`lab_amplitudes`: $A_{i,q}$, $q = xx, yy, xy$)
↓
for each angle Θ: V and H amplitudes and coefficients (`combine`)
↓
normalisation $1/(8\pi^2)$ × units factor (+ angular average if requested)
↓
write result file + `out_plot`

3.4.1 Low-level utilities

`TokenReader` reads stdin like C `scanf` (whitespace-separated tokens; interactively or from a pipe).

`euler_matrices`
ZYZ Euler matrices $T(\varphi, \theta, \psi)$ of equation (2.9), vectorised.

`read_beta` reads the 27 components and symmetrises the tensor (Section 2.8.1).

`rotate_needed`
rotates only the combinations $(j, k) \in \{(xx), (yy), (xy)\}$ needed for the polarisations (equation (2.8)).

`pol_sums` builds the six coefficients a, b, c from the amplitudes (equations (2.14)to (2.19)).

3.4.2 Orientation and angle tables

- `Orientations`: the $20 \times 10 \times 20 = 4000$ orientations of the average (2.6): trapezoid rule in φ and ψ, Gauss–Legendre in $\cos\theta$ (`theta_rule`); the weights sum exactly to $8\pi^2$ (Section 2.8.2).
- `AngleTable`: $\cos\Theta$ and $\sin\Theta$ for every requested angle.

3.4.3 Rotation, combination and finalisation

`lab_amplitudes`
rotates the tensor to the laboratory frame for the 4000 orientations. It does not depend on Θ (a single molecule has no phase factor) and is therefore computed only *once*, whatever the number of angles.

`combine` for one angle, forms the V amplitude (X component) and the H amplitude $\cos\Theta\, A_Y - \sin\Theta\, A_Z$, and the orientational sums.

`finalize` rotation followed by `combine` for one angle (used by the tests).

`compute` loops over the angles and returns `acc[nang, 2, 5]`.

3.4.4 Numerical parameters

```
N_PHI, N_THETA, N_PSI = 20, 10, 20     # 4000 orientations
FMT = "%f"                             # "%.6e" when the units multiplier is not 1
```

For a single molecule the integrand is a trigonometric polynomial of degree ≤ 6, so this grid is *exact* up to rounding (the program reproduces the analytic values above to 10^{-10}): no convergence study and no automatic sizing (Section 2.8.3) are needed. A run with 180 angles takes a fraction of a second.

3.4.5 Outputs

- **Result file** (3rd argument): header with the β tensor, the units convention, the coefficients and the ratios:

```
******************************************************
  Intensity units: raw coherent-sum average |sum beta exp(iq.r)|^2, radiative prefactor NOT included (
     multiply by (|K|^2/4pi)^2 = 1.206331e-09 for r^2|E|^2)
  Units convention: raw (module default) ; PYSHS_I_SCALE = 1.000000e+00 ; total multiplier = 1.000000e+00

 POLAR PLOT - CONFIGURATION AT 90.000000  deg

nombre de dipole = 1
************************
I2v=0.666667  I4v=-0.000000
I2h=-0.000000  I4h=-0.000000
************************
av=0.142857   bv=0.171429   bv1=0.000000  bv2=0.000000  cv= 0.028571
ah=0.028571   bh=0.057143   bh1=0.000000  bh2=-0.000000  ch= 0.028571
```

- `out_plot`, written in the directory of the output file: for the polar modes three columns γ (radians), I_V, I_H; for `angle_scattering` five columns Θ (degrees), $I_H(0°)$, $I_H(90°)$, $I_V(0°)$, $I_V(90°)$.

3.4.6 Code $\leftrightarrow$ equation map

Equation	Code function / variable
(2.9) Euler matrix T	`euler_matrices`
(2.8) $\beta^{\text{lab}} = TTT\beta^{\text{mol}}$	`rotate_needed`, `lab_amplitudes`
V, H amplitudes, equations (2.12)–(2.13)	`combine`
(**??**) coefficients a, b, c, b_1, b_2	`pol_sums`
(**??**) isotropic average	weights `ori.w` and factor $1/(8\pi^2)$

3.4.7 Limitations and practical notes

1. The program describes *uncorrelated* molecules. Correlated molecules (supramolecular structures) require the SHS program (Chapter 4); the two give identical results for $N = 1$ (this is tested).

2. No local-field or concentration factor is applied: the intensities are those of a single molecule (multiply by the number of scatterers, in the same units, for an incoherent ensemble).

3. The absolute scale is the `raw` one by default (Section 2.8.4).

Chapter 4

SHS program

4.1 Inputs

1. Hyperpolarizability tensor: same `input_beta` format as HRS (27 components after a header line; symmetrised in its last two indices, Section 2.8.1).
2. Position/orientation file (mesoscopic frame): six columns per line $(\phi, \theta, \psi, x, y, z)$ for each dipole. **Angles in radians.** The number of lines is the number of molecules N; if the file holds more lines than the N entered, the surplus is ignored and a warning is printed.
3. Interactive parameters on stdin: N, wavelength λ (nm), real and imaginary parts of n, then scattering angle(s) according to the keyword.

The generation scripts of the package can build orientation files for spheres, spheroids, cylinders, etc. (see Chapter 10).

4.2 Command line

```
pyshs-optics-shs <keyword> <input_beta> <input_orientation> <output_file> [--legacy]
```

Example (`polarplot_single` at 90°):

```
printf "1000\n800\n1.33\n0.0\n90\n" | \
  pyshs-optics-shs polarplot_single beta.txt orient.txt resultat.txt
```

Environment variables:

- `PYSHS_THREADS=`k: number of worker threads (default = number of CPU cores);
- `PYSHS_LEGACY=1` (or flag `--legacy`): reproduce two defects of the old C code (cross terms of b_{H1}, b_{H2} and missing $8\pi^2$ normalisation of b_1, b_2); it changes no reported quantity (Section 2.8.7);
- `PYSHS_GRID`, `PYSHS_GRID_SCALE`, `PYSHS_GRID_MAX`: automatic orientation grid (Section 2.8.3);
- `PYSHS_UNITS`, `PYSHS_I_SCALE`: intensity units and scale (Section 2.8.4); `PYSHS_ANGLE_RANGE`: angle grid (Section 2.8.5).

Plotting:

```
python3 -m py_shs.graph_angle    out_plot  image.png
python3 -m py_shs.graph_assembly orientation_file  image.png
```

4.3 Example: a spherical shell of dipoles

The generation script `script_sphere.py` (Chapter 10) places N dipoles on a sphere of radius R, each oriented along the outward normal. With the surface tensor $\chi_{zzz} = 1$, $\chi_{zxx} = \chi_{zyy} = 0.3$, $\chi_{xxz} = \chi_{xzx} = \chi_{yyz} = \chi_{yzy} = 0.2$ as the molecular β, the `angle_scattering` mode gives the angular distribution of Figure 4.1 (right). The coherent sum over the shell concentrates the signal in a lobe whose position and width depend on R/λ (compare with the sphere module, Chapter 5, with which this assembly agrees to better than 10^{-3} when $\chi = \sigma\beta$). For a perfectly continuous achiral sphere $I_V(0°)$ and $I_V(90°)$ vanish; the small non-zero values of the figure are due to the discrete sampling of the surface by 400 dipoles.

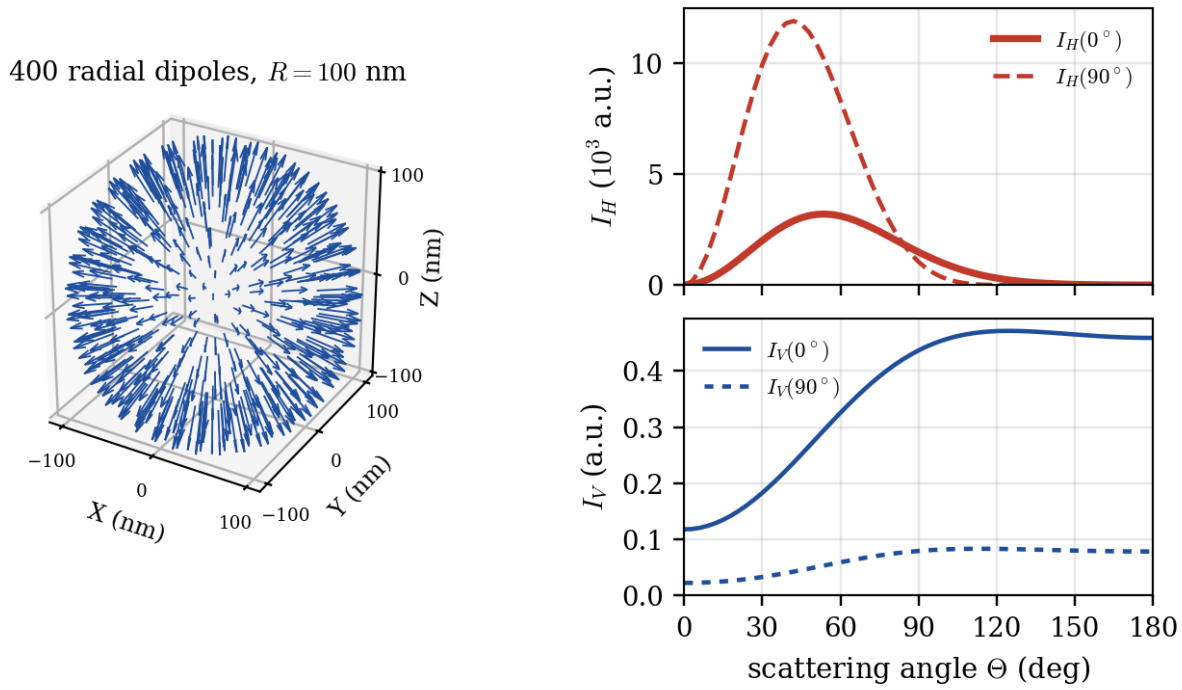


Figure 4.1: SHS of a spherical shell of $N = 400$ radial dipoles ($R = 100$ nm, $\lambda = 800$ nm, $n = 1.33$). Left: the assembly (`out_arrows_gnuplot` / 3-D view of the GUI). Right: $I_H(0°)$, $I_H(90°)$ (top) and $I_V(0°)$, $I_V(90°)$ (bottom) versus Θ (`angle_scattering`); intensities in the `raw` convention.

4.4 Code architecture

The program follows the physical chain of Section 2.3:

`stdin` (interactive or pipe)
↓
read N, λ, $n = n_r + in_i$, angles according to the mode
↓
read β (beta file) and the N dipoles (orientation file)
→ rotate each β into the mesoscopic frame (`rotate_tensors`)
→ store $P[N, 27]$ and positions pos$[N, 3]$
↓
choose the orientation grid from the size of the assembly (`shs_grid`)
↓
loop over scattering angles Θ
for each assembly orientation (chunked, multi-threaded):
accumulate $S = \sum_d \exp(i\,\mathbf{q}\cdot\mathbf{r}_d)\,P_d$ (`accumulate`)
rotate S to the laboratory and form V/H coefficients (`finalize`)
↓
normalisation $1/(8\pi^2)$ (+ angular average if requested)
write result file + `out_plot`

4.4.1 Low-level utilities

`TokenReader`
: Reads stdin like C `scanf` (whitespace-separated tokens; works interactively or from a pipe).

`euler_matrices`
: ZYZ Euler matrices, numerically identical to the C `Txx`...`Tzz`.

`rotate_tensors`
: Implements equation (2.34) via three successive `einsum` calls (molecule → mesoscopic).

`rotate_needed`
: Rotates only the $(j, k) \in \{(xx), (yy), (xy)\}$ combinations needed for the polarisations (avoids rotating all 27 components).

`pol_sums`
: Builds the five coefficients a, b, c, b_1, b_2 from complex amplitudes of the nine A_{XXX}, A_{XYY}, A_{XXY} A_{YXX}, A_{YYY}, A_{YXY} A_{ZXX}, A_{ZYY}, A_{ZXY} and orientation weights (equation (2.58)) according to equations (2.48) to (2.57) followed by the normalisation $1/(8\pi^2)$. The coefficients b_1 and b_2 vanish after the isotropic average (Section 2.8.7); they are kept for completeness.

4.4.2 Orientation and angle tables

- `Orientations`: precomputes the matrices $T(\varphi, \theta, \psi)$ and the weights $w = \Delta\varphi\,\Delta\psi\,w_\theta^{\mathrm{GL}}$ of a grid with N_φ and N_ψ trapezoid points and N_θ Gauss–Legendre points in $\cos\theta$ ($20 \times 10 \times 20 = 4000$ at least; the weights sum exactly to $8\pi^2$); the grid is sized automatically for large assemblies.
- `AngleTable`: for each Θ stores $q_Y = -K\sin\Theta$ and $q_Z = 2k - K\cos\Theta$ (real and imaginary parts separated for absorbing media).

4.4.3 Accumulation and finalisation

`accumulate`
: Builds $\mathbf{q}$ in the mesoscopic frame (equation (E)), evaluates phases, and performs the coherent sum (2.36) as a BLAS matrix product (`float64` real/imaginary parts stored separately).

`finalize` Reconstructs $A = S_r + iS_i$, rotates the needed components to the laboratory, forms V and H amplitudes, and calls `pol_sums`. In legacy mode the historical b_{H1}/b_{H2} cross terms are restored.

`compute` Splits orientations into chunks, runs them with `ThreadPoolExecutor`, reports progress, returns `acc[nang, 2, 5]`.

4.4.4 Numerical parameters

```
PAS = 10                  # minimum orientation grid density
N_PHI, N_THETA, N_PSI     # minimum 20 x 10 x 20 = 4000 orientations
BLOCK = 4096              # dipoles per phase block
TARGET_ELEMS = 1 << 20    # memory cap for (orientations x dipoles) work arrays
```

The orientation grid actually used is chosen from the size of the assembly by `shs_grid.dipole_grid`: $N_\varphi = N_\psi \geq 2q_{\max}r_{\max} + 8$ and $N_\theta = N_\varphi/2$, where $r_{\max}$ is the largest distance of a dipole to the origin (Section 2.8.3); the run log prints it, for instance

```
Orientation grid: 59 x 29 x 59  (max |r| = 600.0, q*rmax = 25.1)
```

The cost grows as $N_\varphi N_\theta N_\psi N_{\text{dipoles}}$ per angle. A coherent chain of dipoles 1.2 μm long is a severe test: the historical $20 \times 10 \times 20$ grid overestimates its signal by 30 % whereas the automatic grid is converged.

BLAS oversubscription is avoided by forcing `OPENBLAS_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, etc., so that only the Python thread pool provides parallelism.

4.4.5 Outputs

- **Result file** (4th argument): header with the β tensor, the units convention, then coefficients and ratios I_2, I_4.
- `out_plot`: table for gnuplot / plotting scripts (γ in radians, I_V, I_H; or Θ in degrees and the four angular intensities).
- `out_arrows_gnuplot`: dipole positions and unit vectors for 3-D visualisation.
- The two side files are written in the *directory of the output file* (not in the current directory); running twice in the same directory overwrites them.

4.4.6 Code ↔ equation map

Equation	**Code function / variable**
(2.35) $\Delta\mathbf{k}^{\text{lab}}$	`petitk`, `grandk`, `AngleTable`
(2.34) $\beta^{\text{mol}} \to \beta^{\text{meso}}$	`rotate_tensors` in `load_dipoles`
(E) $\mathbf{q} = q_Y T_{Y,:} + q_Z T_{Z,:}$	`accumulate` (`kr = ...`)
(2.36) coherent sum S	`accumulate` (matrix product)
Laboratory rotation + A_{ij}	`rotate_needed`
V, H amplitudes	`finalize`
(**??**) coefficients a, b, c, b_1, b_2	`pol_sums`
Isotropic average	weights `ori.w` + factor $1/(8\pi^2)$

4.4.7 Limitations and practical notes

1. **Memory:** all dipoles are loaded (~ 240 bytes/dipole). On a 16 GB machine a practical upper bound is $N \sim 2$–4×10^7.
2. **Orientation grid:** sized automatically; the cost grows as N^3, hence an upper bound $N \leq 80$ (a warning is printed when it is reached; raise it with `PYSHS_GRID_MAX`). With absorbing media the result depends on the origin of the positions (Section 2.8).
3. **Absorbing media:** handled via $\mathrm{Im}(\mathbf{q})$.
4. **Legacy mode:** changes no reported quantity (Section 2.8.7); the default mode is the correct one.
5. **Thread safety:** each chunk writes a distinct slice of S; no race conditions.
6. **Units:** the default is the `raw` convention; use `PYSHS_UNITS=rad` to compare with the continuum modules (Section 2.8.4).

Chapter 5

Sphere Object (`sphere_SHS.py`)

The program `sphere_SHS.py` implements Section 2.4.1: the SHS intensity of a sphere whose surface carries the susceptibility $\chi^{(2)}$, in the NLWKB approximation (or the classical Rayleigh-Gans-Debye one). The exact nonlinear Mie theory is a separate program (Section 2.4.2).

5.1 Inputs

- Surface susceptibility / hyperpolarizability tensor (`input_beta`, same format as HRS; z is the outward normal of the sphere. The tensor is symmetrised in its last two indices (Section 2.8.1).
- NLWKB flag (1 = yes, 0 = classical RGD with a single index).
- Wavelength λ (nm).
- If NLWKB: n_p (particle) and n_m (medium), real and imaginary parts; otherwise a single refractive index n. The classical RGD calculation is the NLWKB one with $n_p = n_m$ (in the GUI, enter the same index twice).
- Sphere radius R (nm).
- Scattering angle(s) or number of angular points.

The interactive parameters are read from stdin in this order:

#	Parameter
1	NLWKB flag (1 or 0)
2	wavelength λ (nm)
3–6	$\mathrm{Re}\, n_p$, $\mathrm{Im}\, n_p$, $\mathrm{Re}\, n_m$, $\mathrm{Im}\, n_m$ (flag = 1); or $\mathrm{Re}\, n$, $\mathrm{Im}\, n$ (flag = 0)
next	radius R (nm)
last	`polarplot_single`: Θ (deg; 0 = forward); `polarplot_integrate`: $\Theta_{\min}$, $\Theta_{\max}$, number of points; `angle_scattering`: number N of intervals between 0° and 180°

5.2 Command line

```
pyshs-optics-sphere <keyword> <input_beta> <output_file>
```

Example (NLWKB, $\lambda = 800$ nm, $n_p = 1.5$, $n_m = 1.33$, $R = 50$ nm, $\Theta = 90°$):

```
printf "1\n800\n1.5\n0.0\n1.33\n0.0\n50\n90\n" | \
  pyshs-optics-sphere polarplot_single chi.txt result.txt
```

Optional environment variables: `PYSHS_UNITS`, `PYSHS_I_SCALE` (units and scale, Section 2.8.4), `PYSHS_GRID`, `PYSHS_GRID_SCALE`, `PYSHS_GRID_MAX` (surface grid, Section 2.8.3), `PYSHS_ANGLE_RANGE` (Section 2.8.5).

5.3 Example

Figure 5.1 shows the result for $\chi_{zzz} = 1$, $\chi_{zxx} = \chi_{zyy} = 0.3$, $\chi_{xxz} = \chi_{xzx} = \chi_{yyz} = \chi_{yzy} = 0.2$ in an index-matched medium ($n_p = n_m = 1.33$). The left panel gives the angular distribution of $I_H(0°)$ of three spheres, each normalised to its own maximum (the maximum, in absolute units, is given in the legend: 2.6×10^{-4}, 0.38 and 18 for $R = 25$, 100 and 250 nm, i.e. a very fast growth with the radius). The distribution is broad for the small sphere, with a maximum at $\Theta = 86°$, and becomes more and more forward-peaked as R/λ increases (maximum at 54° for $R = 100$ nm and 24° for $R = 250$ nm, with secondary maxima for the largest sphere). The signal is zero in the exact forward and backward directions. The right panel gives the polarisation curves at $\Theta = 90°$ for $R = 100$ nm, the signature of the achiral sphere derived in Section 2.4.1: $I_V(\gamma)$ has four maxima at $\gamma = 45° + k\,90°$ and vanishes for $\gamma = 0°, 90°, \ldots$ ($a_V = c_V = 0$, $I_4^V = -1$), whereas $I_H(\gamma)$ is a smooth curve with period 180°.

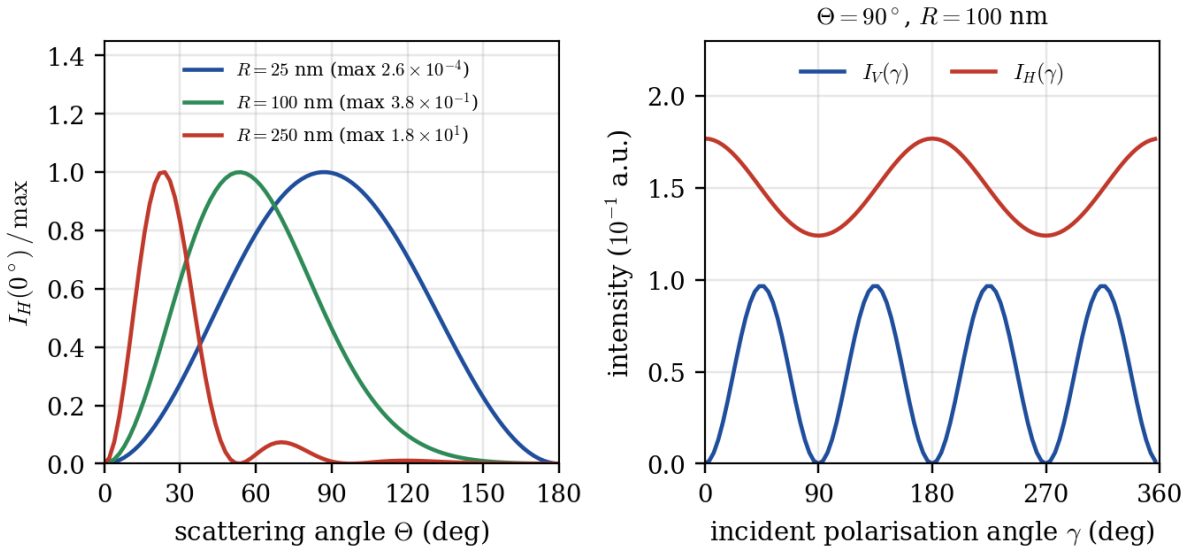


Figure 5.1: Sphere module. Left: $I_H(0°)$ versus Θ for $R = 25$, 100 and 250 nm, each curve normalised to its maximum (`angle_scattering`, $N = 90$). Right: polarisation curves $I_V(\gamma)$, $I_H(\gamma)$ at $\Theta = 90°$ for $R = 100$ nm. $\lambda = 800$ nm, $n_p = n_m = 1.33$, intensities in the `rad` convention.

5.4 Code architecture

`stdin`: flag, λ, indices, R, angles
↓
$k^{\omega} = 2\pi n_p/\lambda$, $K^{2\omega} = 4\pi n_m/\lambda$ (equations (2.60)–(2.61)); units factor (`shs_units`)
↓
read $\chi^{(2)}$, symmetrise (`read_beta`); choose the surface grid (`shs_grid.sphere_pas`)
↓
for each angle Θ:
  `surface_gamma`: $\chi^{(2)}_{IJK}(\varphi_c, \theta_c)$ at every surface point, phase, weights, sum $\rightarrow \Gamma_{IJK}$
  `coefficients_from_gamma`: $a_V, b_V, c_V, a_H, b_H, c_H$
↓
$I_V(\gamma)$, $I_H(\gamma)$ or the four angular intensities; I_2, I_4
write result file + `out_plot`

5.4.1 Low-level utilities

`TokenReader`, `say` stdin reader and message printing.

`euler_matrices` ZYZ matrices $T(\varphi_c, \theta_c, 0)$ giving the local frame of every surface point (equation (**??**)).

`rotate_tensor_pointwise`
rotates the tensor at all surface points at once ($n_x \times n_y \times 3 \times 3 \times 3$ array).

`read_beta` reads and symmetrises the tensor.

5.4.2 Surface integral and coefficients

`surface_gamma` builds the grid (φ_c trapezoid, $\cos\theta_c$ Gauss–Legendre), the phase Φ of equation (2.67) (evaluated as $e^{+i\Delta\mathbf{k}\cdot\mathbf{r}}$, i.e. the complex conjugate: same intensities), the weights $R^2 w_b\, 2\pi/n_x$, and sums the integrand to obtain the dictionary of the 6 components $\Gamma_{XXX}, \Gamma_{XYY}, \Gamma_{XXY}, \Gamma_{YXX}, \ldots$ needed.

`coefficients_from_gamma`
combines the components into $a_V, b_V, c_V, a_H, b_H, c_H$ with the expressions of Section 2.4.1 (no orientational average).

`intensity_VH_extremes`
the four intensities of `angle_scattering`: $I_V(0°) = a_V$, $I_V(90°) = c_V$, $I_H(0°) = a_H$, $I_H(90°) = c_H$.

`write_header`, `report_polar`, `main`
output files and program flow.

5.4.3 Numerical parameters

```
PAS_SURFACE = 40      # minimum: n_y = 40 (cos theta_c), n_x = 2 n_y (phi_c)
```

The grid is enlarged automatically for large spheres: $n_y = \max\big(40, \lceil q_{\max} R/2 \rceil + 20\big)$ with $q_{\max} = 2k^{\omega} + K^{2\omega}$ (Section 2.8.3); it is printed in the log (e.g. `nx = 218, ny = 109` for $R = 4$ μm) and bounded by $n_y \leq 400$ (a warning is printed beyond). With the Gauss–Legendre rule the error is $\sim 10^{-7}$ or less up to $R = 4$ μm, whereas the historical fixed mid-point grid of 40 points was wrong by about 1 % at $R = 300$ nm, 5 % at 600–1000 nm and by orders of magnitude beyond 2 μm (Figure **??**). `PYSHS_THETA_RULE=midpoint` restores the former rule.

5.4.4 Outputs

- **Result file**: banner, mode, indices, wave vectors, radiative prefactor and surface grid, the $\chi^{(2)}$ tensor, the units convention, then the coefficients and ratios:

```
petitk              : 0.011781 + i*0.000000
grandk              : 0.020892 + i*0.000000
Radiative pref.     : |k|^2/(4pi) = 3.473227e-05  (I *= pref^2 = 1.206331e-09)
Surface quadrature  : pas = 40 (nx=80, ny=40)
```

```
  Intensity units: r^2|E_2w|^2/|E_w|^4, radiative prefactor (|K|^2/4pi)^2 = 1.206331e-09 included
  Units convention: rad (module default) ; PYSHS_I_SCALE = 1.000000e+00 ; total multiplier = 1.206331e-09
*******************************************
* Sphere POLAR PLOT - CONFIGURATION AT 90.0  deg
*******************************************
I2v=0.000000  I4v=-1.000000
I2h=0.029266  I4h=0.000107
av=0.000000   bv=0.024192   cv=0.000000
ah=0.014601   bh=0.028360   ch=0.013771
```

- `out_plot` (in the directory of the output file): γ (rad), I_V, I_H for the polar modes; Θ (deg), $I_H(0°)$, $I_H(90°)$, $I_V(0°)$, $I_V(90°)$ for `angle_scattering` ($N+1$ angles from 0° to 180°).

5.4.5 Code ↔ equation map

Equation	Code
(2.60), (2.61) wave vectors	`petitk`, `grandk`
(**??**) surface frame $T(\varphi_c, \theta_c, 0)$	`euler_matrices`, `rotate_tensor_pointwise`
(2.67) phase Φ	`phase` in `surface_gamma` (opposite sign)
(2.65) Γ_{IJK}	`surface_gamma`
coefficients $a_V, \ldots, c_H$	`coefficients_from_gamma`
$(\lvert K\rvert^2/4\pi)^2$ and `PYSHS_I_SCALE`	`rad_pref2` (from `shs_units`)

5.4.6 Limitations and practical notes

1. The NLWKB treatment of the fundamental field is approximate; for an index contrast use the Mie program (Section 2.4.2), which is exact (Figure 2.5).

2. The surface tensor is the *full* 27-component tensor in the surface frame; chiral components are accepted here (they are ignored by the Mie program).

3. With complex indices the result depends on the origin of the positions; the sphere is centred at the origin.

4. For a small sphere the intensity grows as R^6 (measured exponent ≈ 6 for $R \lesssim 40$ nm at 800 nm): the dipolar term vanishes for a centrosymmetric particle and the signal comes from the first retardation correction. The absolute scale follows Section 2.8.4.

Chapter 6

Cylinder Object (`cylinder_SHS.py`)

The program `cylinder_SHS.py` implements Section 2.5: a finite cylinder of radius R and length L whose *lateral* surface carries the susceptibility $\chi^{(2)}$, averaged over all the orientations of its axis.

6.1 Inputs

- Surface susceptibility tensor (`input_beta`; local frame $x = -$axis direction, $y =$ azimuthal tangent, $z =$ radial normal, Section 2.5).
- Wavelength λ (nm).
- n_p (real, imag), n_m (real, imag) (NLWKB wave vectors, equations (2.84)–(2.85)).
- Cylinder radius R (nm) and length L (nm).
- Scattering angle(s) or number of points.

Order on stdin: λ, $\mathrm{Re}\, n_p$, $\mathrm{Im}\, n_p$, $\mathrm{Re}\, n_m$, $\mathrm{Im}\, n_m$, R, L, then the angle(s) as in Chapter 5 (there is no NLWKB flag).

6.2 Command line

```
pyshs-optics-cylinder <keyword> <input_beta> <output_file>
```

Example ($R = 50$ nm, $L = 300$ nm, $\Theta = 90°$):

```
printf "800\n1.5\n0.0\n1.33\n0.0\n50\n300\n90\n" | \
  pyshs-optics-cylinder polarplot_single chi.txt result.txt
```

The environment variables are those of Chapter 5; `PYSHS_THETA_MESO_LEGACY=1` restores the truncated polar angle 1.570796 of the former C code (effect $\sim 10^{-7}$).

6.3 Example

Figure 6.1 shows the effect of the length at fixed radius ($R = 25$ nm). The maximum of $I_H(0°)$ grows by a factor of about 20 between $L = 100$ and 1000 nm (1.2×10^{-3}, 6.1×10^{-3} and 2.3×10^{-2} for $L = 100$, 300 and 1000 nm: more radiating surface) and the maximum of the angular distribution moves from $\Theta = 88°$ to 72° and 68°, towards the forward direction, as expected for an object whose length becomes comparable to the wavelength. In the polarisation curves at $L = 300$ nm the V signal keeps four maxima at $\gamma = 45° + k\,90°$ but is no longer zero for $\gamma = 0°$ ($a_V, c_V \neq 0$), and I_H varies markedly with γ: for a non-spherical object $a_V = \langle|\Gamma_{XXX}|^2\rangle$ does not vanish by symmetry as it does for the sphere (Section 2.4.1).

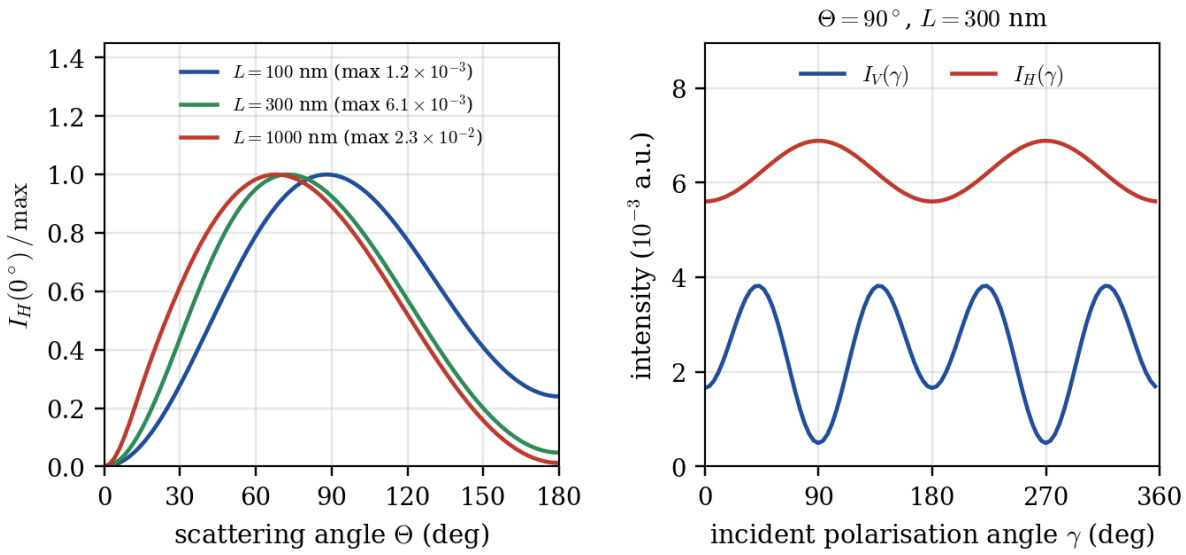


Figure 6.1: Cylinder module. Left: $I_H(0°)$ versus Θ for $L = 100$, 300 and 1000 nm ($R = 25$ nm), each curve normalised to its maximum. Right: polarisation curves at Θ = 90° for $L = 300$ nm. $\lambda = 800$ nm, $n_p = n_m = 1.33$, `rad` convention.

6.4 Code architecture

`stdin`: λ, indices, R, L, angles
↓
wave vectors, units factor, read and symmetrise $\chi^{(2)}$
↓
choose $(n_\beta, n_\varphi, n_\theta)$ from R and L (`shs_grid.cylinder_grid`)
↓
`compute`: surface tensors $\chi^{(2)}_{i'j'k'}(\theta_c)$ for the n_β azimuths (once)
for every orientation (φ, θ) of the axis (trapezoid × Gauss):
lab-frame tensors, phases of the n_β strips, analytic axial sinc factor for all angles
amplitudes A_{iq}, orientational sums of a, b, c for V and H
↓
normalisation $1/\sum w$, radiative prefactor; report

6.4.1 Functions

`euler_matrix`, `rotate_tensor`
: Euler matrix and rotation of a tensor (equations (2.93), (2.94)).

`read_ki`
: reads and symmetrises the surface tensor.

`theta_rule`
: Gauss–Legendre (default) or mid-point nodes and weights in θ.

`_csinc`
: $\sin x/x$ for complex arguments, with a series near 0.

`compute`
: the whole calculation; vectorised over the azimuthal strips and over all angles; returns the coefficients and the sum of the orientation weights.

`write_header`, `report_polar`, `main`
: output and flow. The report titles state the angle (single mode) or the angular range and number of points (average).

6.4.2 Numerical parameters

```
N_BETA, N_PHI, N_THETA = 10, 10, 10     # minimum sizes
THETA_MESO = pi/2                       # polar angle of the surface frame
```

The sizes actually used are $n_\beta = \max(12, \lceil q_{\max} R \rceil + 14)$ and $n_\varphi = n_\theta = \max(12, \lceil q_{\max} \sqrt{R^2 + L^2/4} \rceil + 12)$; the axial integral has no grid (equation (2.95)). The log prints, e.g.,

```
Grid: azimuth 17, orientation 20 (phi) x 20 (theta); axial integral analytic
```

For a rod of $L = 2$ μm the former 10×10 grid gave results 3 times too small (Figure **??**); the automatic grid is converged to $\sim 10^{-7}$. Bounds: $n_\beta \leq 128$, $n_\varphi = n_\theta \leq 160$.

6.4.3 Outputs

Same files as Chapter 5 (the header gives the grid: `Grid: azimuth, orientation`); the report title of `polarplot_integrate` is `average over` $\Theta_{\min}$ `...` $\Theta_{\max}$ `deg (`n `points)`.

6.4.4 Code ↔ equation map

Equation	Code
(2.86), (2.94) surface frame	`Pb` in `compute` ($T(\beta, \pi/2, 0)$)
(2.87), (2.93) laboratory frame	`full` (`einsum` with T)
(2.92) phase of a strip, (2.95) axial integral	`ph`, `fz`
(2.96) orientational average	loops over `n_phi`, `n_theta`, weights `th_w`, `wsum`
coefficients a, b, c (V, H)	`pol_sums` in `compute`

6.4.5 Limitations and practical notes

1. Lateral surface only (no end caps): valid for $L \gg R$; for $L \sim R$ the caps matter. The surface is then *open*, which has a measurable consequence. Write $A = \chi_{zzz}$, $B = \chi_{zxx}$, $C = \chi_{xxz}$ for the three constants of an isotropic achiral surface. In the laboratory frame Γ_{XXX} is the surface integral of $(A - B - 2C)\, n_X^3 + (B + 2C)\, n_X$ times $e^{i\Phi}$, and Φ does not depend on X. For a *closed* surface the integral of the term in n_X vanishes (divergence theorem), so that for the particular tensor $A = B + 2C$ the sphere, the ellipsoid and the box give exactly $a_V = c_V = 0$ for any size and orientation (checked numerically to 10^{-30}). The cylinder gives non-zero values for the same tensor ($a_V / b_V \approx 0.14$ for $R = 25$ nm, $L = 300$ nm): this is the contribution of the missing end caps.

2. The calculation is the orientational average of an isotropic suspension of identical cylinders (the average is incoherent; there is no interference between different cylinders).

3. The cylinder is centred at the origin.

Chapter 7

Ellipsoid Object (`ellipsoid_SHS.py`)

The program `ellipsoid_SHS.py` implements Section 2.6: a spheroid with equatorial radius a and half-length c along the symmetry axis ($c = L/2$), parameterised by $x = a\sin\alpha\cos\beta$, $y = a\sin\alpha\sin\beta$, $z = c\cos\alpha$ with $\alpha \in [0, \pi]$ and $\beta \in [0, 2\pi]$. The local surface frame of the $\chi^{(2)}$ tensor is the meridian tangent (x), the azimuthal tangent (y) and the outward normal (z), equation (2.105) and Figure 2.7. The surface integration is coherent (amplitudes); the orientational average is incoherent (intensities).

7.1 Inputs

- Surface $\chi^{(2)}$ tensor (`input_beta`).
- NLWKB flag (1/0).
- Wavelength, n_p and n_m (or a single n).
- Equatorial radius a (nm) and *length* $2c$ (nm).
- Scattering angle(s) or number of points.

Order on stdin: flag, λ, indices as in Chapter 5, a, $2c$, then the angle(s).

7.2 Command line

```
pyshs-optics-ellipsoid <keyword> <input_beta> <output_file>
```

Example ($a = 50$ nm, $2c = 300$ nm, $\Theta = 90°$):

```
printf "1\n800\n1.5\n0.0\n1.33\n0.0\n50\n300\n90\n" | \
  pyshs-optics-ellipsoid polarplot_single chi.txt result.txt
```

7.3 Example

Figure 7.1 follows a prolate spheroid of fixed equatorial radius ($a = 50$ nm) as its half-length grows. For $c = a$ the module gives the sphere (this is tested to better than 3×10^{-3}). Lengthening the particle increases the maximum of $I_H(0°)$ by a factor of about 7 (1.3×10^{-2}, 3.8×10^{-2} and 9.3×10^{-2} for $c = 50$, 100 and 250 nm) and moves the maximum of the angular distribution from $\Theta = 78°$ to 68° and 56°, towards the forward direction. The polarisation curve $I_V(\gamma)$ at $c = 100$ nm is close to that of the sphere (four maxima, minimal signal at $\gamma = 0°$): a_V and c_V are not zero for an ellipsoid but remain very small for a compact particle ($a_V/b_V \approx 3 \times 10^{-3}$ at $\Theta = 90°$ for $2c = 400$ nm, with the tensor of this example).

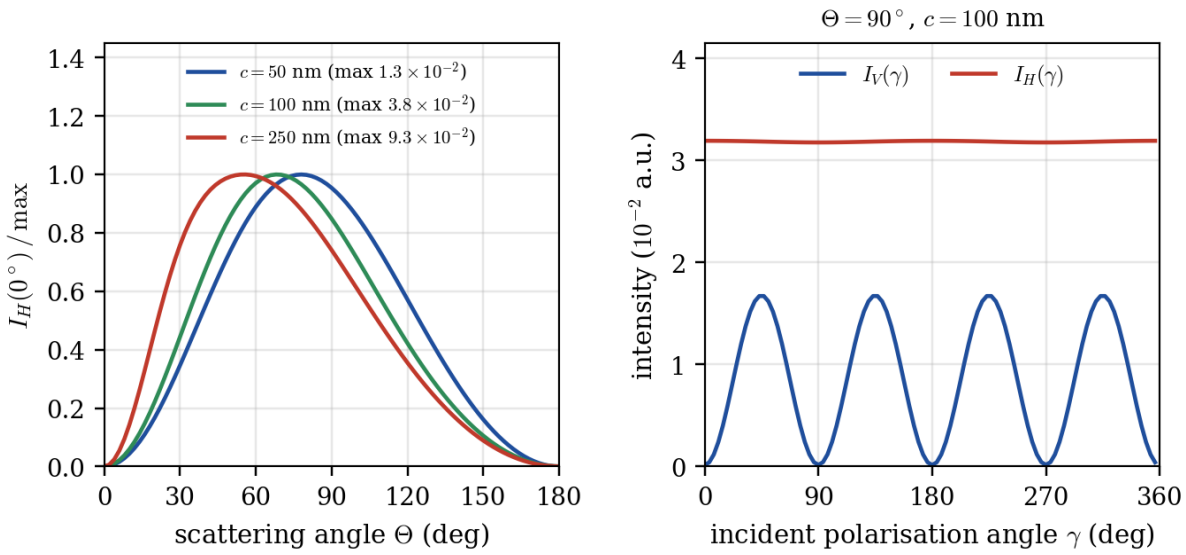


Figure 7.1: Ellipsoid module ($a = 50$ nm). Left: $I_H(0°)$ versus Θ for $c = 50$, 100 and 250 nm, each curve normalised to its maximum. Right: polarisation curves at $\Theta = 90°$ for $c = 100$ nm. $\lambda = 800$ nm, $n_p = n_m = 1.33$, `rad` convention.

7.4 Code architecture

`stdin`: flag, λ, indices, a, $2c$, angles
↓
wave vectors, units, read and symmetrise $\chi^{(2)}$; grid from $\max(a, c)$ (`shs_grid.ellipsoid_grid`)
↓
`build_surface`: surface points $\mathbf{r}'$, weights $aN\, w_\alpha\, \Delta\beta$ and tensors $\chi^{(2)}_{i'j'k'}$ in the particle frame
↓
`compute_coefficients`: for every axis orientation (φ, θ):
lab-frame (xx, yy, xy) tensors of all points at once, phases for all angles, amplitudes
orientational sums $\rightarrow a, b, c$ (V, H) for every Θ
↓
normalisation $1/\sum w$, units factor; report

7.4.1 Functions

`euler_zyz`, `rotate_tensor`
: Euler matrix, tensor rotation.

`read_ki`
: reads and symmetrises the surface tensor.

`build_surface`
: vectorised construction of the surface: Gauss–Legendre nodes in $u = \cos\alpha$, uniform β; local frames $S = [\mathbf{t}, \mathbf{e}_\beta, \mathbf{n}]$; $\chi^{(2)}$ in the particle frame (equation (2.106)).

`compute_coefficients`
: orientational average for a list of angles, vectorised over the surface points and the angles; only the (xx, yy, xy) components needed are rotated.

`theta_rule`
: Gauss–Legendre or mid-point nodes in θ.

`write_header`, `polar_output`, `main`
: output and flow.

7.4.2 Numerical parameters

```
N_ALPHA, N_BETA = 30, 20       # surface grid (minimum)
N_PHI, N_THETA  = 20, 20       # orientation grid (minimum)
```

With $\ell = \max(a, c)$ and $q_{\max} = 2k^{\omega}+K^{2\omega}$: $n_\alpha = \max(30, \lceil q_{\max}\ell/2\rceil+20)$, $n_\beta = \max(20, \lceil q_{\max}\ell\rceil+16)$ and $n_\varphi = n_\theta = \max(20, \lceil q_{\max}\ell\rceil + 12)$, bounded by 200, 300 and 160 (Section 2.8.3). The log prints the grid, e.g. `surface 30 x 23, orientations 20 x 20`. The former fixed grid was wrong by 20 % for $a = 100$ nm and $2c = 2$ μm and by a factor ~ 3000 at backward angles for $a = 500$ nm, $2c = 1$ μm; the automatic grid is converged to $\sim 10^{-7}$.

7.4.3 Outputs

Same files as Chapter 5; the header reports a, $2c$ and the quadrature, e.g.:

```
  equatorial radius a = 50.000000 nm   length 2c = 300.000000 nm
  petitk  (k_omega)   = 0.011781 + i*0.000000
  grandk  (k_2omega)  = 0.020892 + i*0.000000
  Radiative pref.     : |k|^2/(4pi) = 3.473227e-05  (I *= pref^2 = 1.206331e-09)
  Quadrature          : surface 30 x 23, orientations 20 x 20
```

7.4.4 Code ↔ equation map

Equation	Code
(2.103), (2.104) geometry and area element	`r`, `w` in `build_surface`
(2.105), (2.106) frame and tensor	`S`, `P` in `build_surface`
(2.107) Γ_{IJK} and phase (2.92)	`ph`, `A` in `compute_coefficients`
(2.108) orientational average	loops over `n_phi`, `n_theta`, `wori`

7.4.5 Limitations and practical notes

1. Body of revolution only (no ψ average): a triaxial ellipsoid is not available.
2. The surface frame follows the geometry (meridian/azimuth); a tensor that is not isotropic in the surface plane therefore describes an anisotropy along the meridians.
3. The ellipsoid is centred at the origin; the same notes on absorbing media as in Chapter 5 apply.

Chapter 8

Cube / Box Object (`cube_SHS.py`)

The program `cube_SHS.py` implements Section 2.7: a rectangular box of edges $a \times b \times c$ (a cube if $a = b = c$). The six flat faces have normals $\pm x$, $\pm y$, $\pm z$ and carry the susceptibility $\chi^{(2)}$ in the face frames of equation (2.110) (Figure 2.8). Face integrals of the optical phase are performed analytically:

$$\int_{\text{face}} e^{i\Phi}\,\mathrm{d}S = \text{Area}\, e^{i\Phi_{\text{centre}}}\,\text{sinc}(X_u/2)\,\text{sinc}(X_v/2).$$

The orientational average uses the periodic trapezoid rule on φ and ψ and Gauss–Legendre nodes on $\cos\theta$.

8.1 Inputs

- Surface $\chi^{(2)}$ tensor (`input_beta`).
- NLWKB flag (1/0).
- Wavelength, n_p and n_m (or a single n).
- Edges a, b, c (nm).
- Scattering angle(s) or number of points.

Order on stdin: flag, λ, indices as in Chapter 5, a, b, c, then the angle(s).

8.2 Command line

```
pyshs-optics-cube <keyword> <input_beta> <output_file>
```

Example (cube of 200 nm, $\Theta = 90°$):

```
printf "1\n800\n1.5\n0.0\n1.33\n0.0\n200\n200\n200\n90\n" | \
  pyshs-optics-cube polarplot_single chi.txt result.txt
```

8.3 Example

Figure 8.1 shows cubes of 100, 200 and 400 nm. The maximum of $I_H(0°)$ increases by a factor of about 430 between 100 and 400 nm (3.7×10^{-2}, 0.91 and 16) and moves from $\Theta = 70°$ to 42° and 22°. The smallest cube has a smooth distribution; for $a = 400$ nm the signal is strongly forward-peaked and shows diffraction minima (near 50° and 105°) coming from the coherent addition of the six faces and their sinc factors. The polarisation curves at $\Theta = 90°$ for $a = 200$ nm are close to those of the sphere ($a_V/b_V \approx 1\%$).

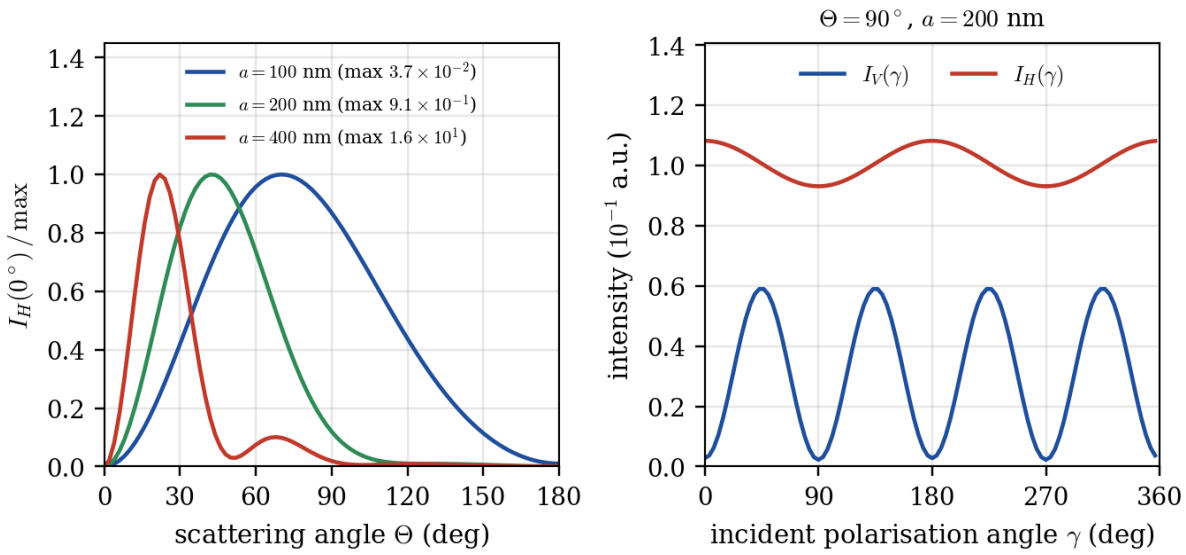


Figure 8.1: Box module. Left: $I_H(0°)$ versus Θ for cubes of edge $a = 100$, 200 and 400 nm, each curve normalised to its maximum. Right: polarisation curves at $\Theta = 90°$ for $a = 200$ nm. $\lambda = 800$ nm, $n_p = n_m = 1.33$, `rad` convention.

8.4 Code architecture

`stdin`: flag, λ, indices, a, b, c, angles
↓
wave vectors, units, read and symmetrise $\chi^{(2)}$; grid from d (`shs_grid.box_grid`)
↓
`build_faces`: for each face: centre $\mathbf{r}_0$, edge vectors $\mathbf{u}, \mathbf{v}$, area and tensor $\chi^{(2)}$ in the particle frame
↓
`compute_coefficients`: all orientations (φ, θ, ψ) in batches
lab-frame tensors of the 6 faces, phases and sinc factors for all angles, amplitudes
orientational sums → a, b, c (V, H)
↓
normalisation, units factor; report

8.4.1 Functions

`euler_zyz`, `rotate_tensor`, `cross3`
Euler matrix, tensor rotation, cross product.

`read_ki` reads and symmetrises the surface tensor.

`gauss_legendre` nodes and weights in $\cos\theta$.

`csinc`, `csinc_array`
$\sin x/x$ for complex arguments (scalar and array), with a series near 0.

`build_faces` the six faces with the frames of equation (2.110).

`compute_coefficients`
orientational average, vectorised over the orientations (processed in batches to bound the memory) and over the angles.

8.4.2 Numerical parameters

```
N_PHI = N_THETA = N_PSI = 24     # minimum orientation grid
```

The faces are exact; only the orientation grid matters: $N = \max(24, \lceil q_{\max} d \rceil + 12)$ with $d = \frac{1}{2}\sqrt{a^2+b^2+c^2}$ (Section 2.8.3), bounded by $N \leq 80$ (cost $\propto N^3$). The fixed 24^3 grid was wrong by 8 % at $a = 1000$ nm and 6 % at 1500 nm; the automatic grid gives $\lesssim 10^{-3}$ over this range (Figure **??**). A 200-nm cube takes a fraction of a second per angle, a 1.5-μm cube a few seconds.

8.4.3 Outputs

Same files as Chapter 5; the header reports the edges and the quadrature:

```
box edges  a = 200.000000 nm   b = 200.000000 nm   c = 200.000000 nm
petitk  (k_omega)   = 0.011781 + i*0.000000
grandk  (k_2omega)  = 0.020892 + i*0.000000
Radiative pref.     : |k|^2/(4pi) = 3.473227e-05  (I *= pref^2 = 1.206331e-09)
Quadrature          : exact face integrals, orientations 24 (phi) x 24 (cos theta, Gauss-Legendre) x 24 (
   psi)
```

8.4.4 Code ↔ equation map

Equation	Code
(2.109), (2.110) faces and frames	`build_faces`
(2.111) face integral with sinc factors	`Ff` (`csinc_array`) in `compute_coefficients`
(2.92) phase of a face	`exp(1j*(qz*zc+qy*yc))`
(2.112) orientational average	`Tall`, weights `wori_all`

8.4.5 Limitations and practical notes

1. The box has sharp edges and corners, which are not modelled specifically (only the surface susceptibility of the faces).
2. Vectorised batches use temporary arrays of about 2×10^6 elements; very large numbers of angles are processed in more batches.
3. The cube is centred at the origin.

Chapter 9

Graphical User Interface

9.1 Launch

Once the package is installed (Section 1.3), start the GUI from a terminal, from any directory:

```
pyshs-optics-gui
```

or, equivalently, with `python3 -m py_shs` (`python -m py_shs` on Windows). If you installed Py_SHS in a virtual or conda environment, activate it first. The GUI requires `tkinter` (see the platform notes of Section 1.3); `matplotlib` and `scipy` (Mie calculation) are installed by `pip`.

The GUI is a front end for the command-line programs: it writes the input files, runs the selected program (as `python -m py_shs.<module>`, with the parameters sent on stdin and the environment variables set from the interface), displays the output and the plots, and lets you export them.

Working directory. There is no `Work/` directory in the installed package. The GUI keeps its files in a directory called `PySHS_Work`, created automatically in your home folder at the first launch:

System	Default location
Linux	`/home/<user>/PySHS_Work`
macOS	`/Users/<user>/PySHS_Work`
Windows	`C:\Users\<user>\PySHS_Work`

Each calculation is run in a temporary sub-directory `PySHS_Work/pyshs_*` (input files, output file, `out_plot`, ...); the intermediate files of the orientation generators and of the Gaussian converter (Chapter 10) are also written in `PySHS_Work`. Run directories older than 7 days are removed automatically each time a calculation is launched; only the directories created by the GUI (name starting with `pyshs_`) are ever deleted, so keep the results you want to preserve by exporting them (Section 9.6) or by copying them elsewhere.

To use another location, set the environment variable `PYSHS_WORK` before starting the GUI:

```
export PYSHS_WORK=/path/to/my/calculations      # Linux / macOS
pyshs-optics-gui

set PYSHS_WORK=D:\calculations                  # Windows (Command Prompt)
pyshs-optics-gui
```

Uninstalling the package with `pip uninstall` does not remove this directory.

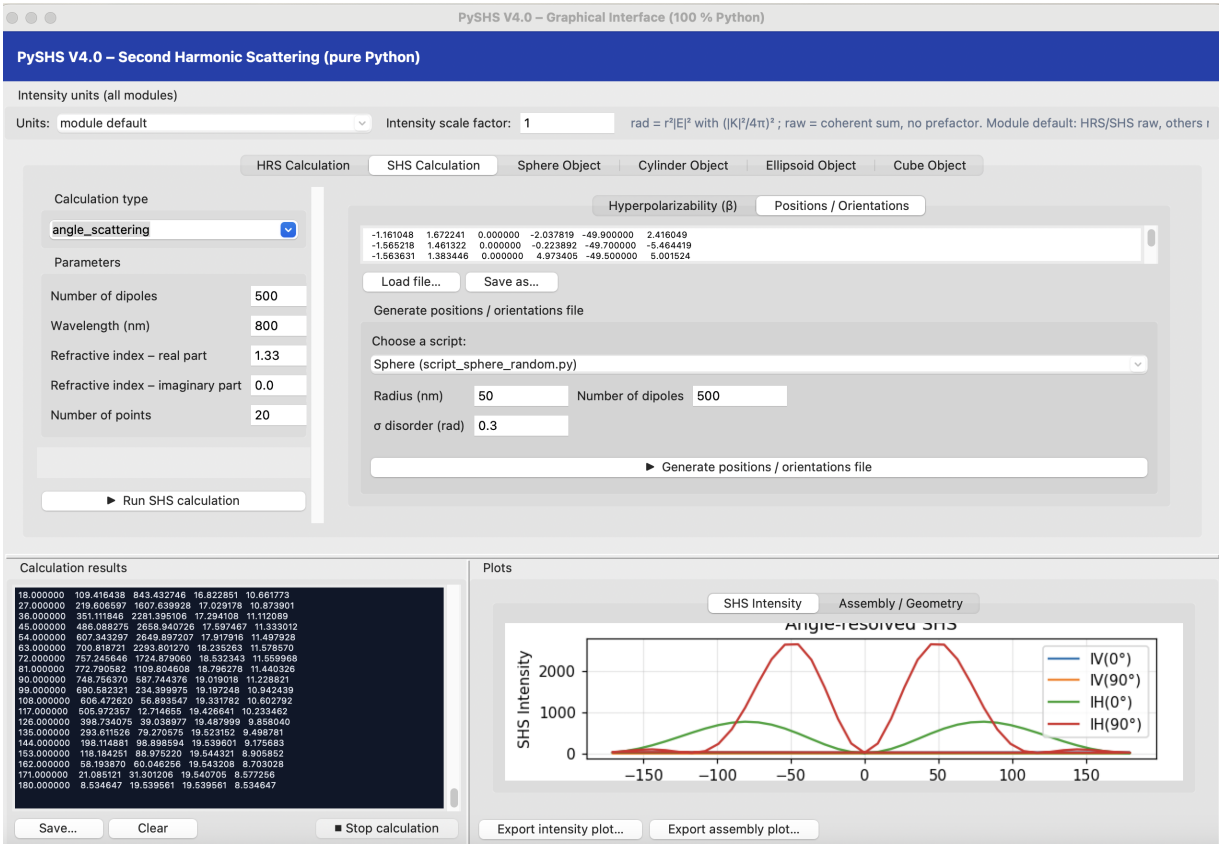


Figure 9.1: a screenshot of the GUI

9.2 Common layout

- **Intensity units (all modules)**, at the top of the window: a selector (*module default*, *rad*, *raw*) and an *Intensity scale factor*. They apply to every module and set `PYSHS_UNITS` and `PYSHS_I_SCALE` for the run (Section 2.8.4). The same scale-factor entry also appears in the Sphere tab.
- **Module tabs**: *HRS Calculation*, *SHS Calculation*, *Sphere Object*, *Cylinder Object*, *Ellipsoid Object*, *Cube Object*.
- **Results area**: a text console (the log of the run, which includes the grid actually used and any warning) and two plot tabs, *SHS Intensity* and *Assembly / Geometry*, with the buttons *Export intensity plot...* and *Export assembly plot...* and a *Stop calculation* button.

9.3 HRS tab

- Hyperpolarizability tensor: a text area holding the 27 components in the `input_beta` file format, with the buttons *Load example* ($\beta_{zzz} = 1$), *Load file...* and *Load Gaussian file...* (conversion of a Gaussian log, Chapter 10).
- Wavelength, refractive index, calculation mode (`polarplot_single`, `polarplot_integrate`, `angle_scattering`) and the associated angle(s).

9.4 SHS tab

Two sub-tabs:

- *Hyperpolarizability (β)*: the molecular tensor (text area, *Load example* and *Load file...*), as in the HRS tab.
- *Positions / Orientations*: a text area with one line per molecule $(\varphi, \theta, \psi, x, y, z)$, which can be edited, loaded (*Load file...*), saved (*Save as...*) or produced by one of the generators (*Generate positions/orientations file*: sphere, spheroid, cylinder; Chapter 10).

The orientation grid is sized automatically from the assembly (Section 2.8.3); the log shows it.

9.5 Sphere, Cylinder, Ellipsoid and Cube tabs

Each tab has the geometric parameters of the object (radius; radius and length; equatorial radius and length; three edges), the wavelength, the particle and medium refractive indices (real and imaginary parts), the surface susceptibility tensor (text area with *Load example* and *Load file...*) and the mode and angle(s).

- The **Sphere** tab has two run buttons: the NLWKB/RGD theory (`sphere_SHS.py`; the NLWKB flag is always sent as 1: the classical RGD model is obtained with $n_p = n_m$) and the **Mie theory** (`shs_mie.py`). For the Mie run the tensor can be entered as the three constants $(\chi_{rrr}, \chi_{rtt}, \chi_{trt})$ (sub-tab *Mie isotropic*) instead of the full tensor.
- Intensity units and scale factor: see the common layout above.

9.6 Results and export

The *SHS Intensity* tab shows $I_V(\gamma)$ and $I_H(\gamma)$ as a polar plot (polar modes) or the four intensities versus Θ (angle mode; $\Theta \in [0°, 180°]$ in every module, Section 2.8.5). The unit of a polar file is decided once for the whole file (the programs write γ in radians). The *Assembly / Geometry* tab shows the dipole assembly (SHS) or the object. The plots can be saved as images.

Chapter 10

Additional scripts

10.1 Orientation / position generators (SHS)

All the generators write a six-column file, one line per dipole, in the format read by `SHS.py` (Chapter 4): the Euler angles φ, θ, ψ (radians) and the position x, y, z (nm). The questions are asked on stdin (the GUI sends them), the file name is the only argument and the file is written wherever that path points (the GUI uses its `PySHS_Work` directory):

```
python3 -m py_shs.script_sphere          out_orient    # radius, N
python3 -m py_shs.script_sphere_random   out_orient    # radius, N, sigma
python3 -m py_shs.script_spheroid_random out_orient    # a, c, N, sigma
python3 -m py_shs.script_cylinder_random out_orient    # R, L, N_circ, N_length, sigma
```

`script_sphere.py` N dipoles quasi-uniformly distributed on a sphere of radius R (Fibonacci spiral with a random rotation), each oriented along the outward normal ($\psi = 0$), the frame used by `sphere_SHS.py`.

`script_sphere_random.py` the same with a Gaussian disorder of standard deviation σ (rad) added to φ and θ.

`script_spheroid_random.py` dipoles on a spheroid of equatorial radius a and polar semi-axis c (symmetry axis z, as in `ellipsoid_SHS.py`), placed with a *uniform surface density* (golden-angle spiral with the latitude spacing taken from the cumulative area), oriented along the normal with the meridian-tangent frame of equation (2.105), plus optional disorder σ.

`script_cylinder_random.py` dipoles on the lateral surface of a cylinder of radius R and length L: N_{circ} dipoles around the circumference and N_{length} along the length (the total is $N_{\mathrm{circ}} N_{\mathrm{length}}$), radial orientation ($\theta = \pi/2$), the frame of `cylinder_SHS.py`, plus optional disorder.

Each script also writes a 3-D picture of the assembly, `<file>.png`, when `matplotlib` is available (the picture is optional: without it only the dipole file is written; the dipole file itself is never overwritten by the picture).

A noise σ on φ is a real tilt of $\sigma \sin\theta$ only, so the angular spread is larger near the poles of the normal than near the equator. For the sphere and the spheroid the assemblies reproduce the continuum modules through $\chi = \sigma\beta$ (Section 2.8.4): with $\sigma = N/\mathrm{area}$ the SHS intensity of the assembly equals σ^2 times that of `sphere_SHS.py` or `ellipsoid_SHS.py`, to better than 10^{-2} for a few thousand dipoles.

10.2 Gaussian → PySHS tensor

```
python3 -m py_shs.script_gauss2PySHS  path/to/job.log  beta_from_gauss \
        [--marker TEXT] [--occurrence K] [--offset N]
```

The script looks for the line whose first word is the marker (default `Beta(-2w;w,w)`), then reads 18 consecutive lines starting 8 lines after the marker (default offset) and takes the second word of each. The 18 values are the components $\beta_{i,jk}$ with (jk) in the order xx, xy, yy, xz, yz, zz and, for each, $i = x, y, z$; they are expanded into the 27-component tensor symmetric in the last two indices (the intrinsic permutation symmetry assumed by PySHS, not the Kleinman symmetry) and written as a PySHS `input_beta` file in atomic units. It is accessible from the HRS tab via *Load Gaussian file…*.

- If the log contains several blocks (several frequencies), the *first* one is used and the others are listed in a warning: `--occurrence` K selects block number K.
- The values are validated (18 numeric, finite values), and a clear error is given if the layout does not match (for instance a wrong offset).
- `--marker` and `--offset` adapt the script to another block or layout (for instance a static block, if your log prints it in the same layout). The layout assumed here is the one of the original script; it was not checked against every Gaussian version.

10.3 Plotting

```
python3 -m py_shs.graph_angle    out_plot  image.png
python3 -m py_shs.graph_assembly orientation_file  image.png [--positions-first] [--arrow L]
```

`graph_angle.py` plots the four intensities of an `angle_scattering` file. Since $I(\Theta) = I(-\Theta)$, a half range is mirrored to the full $[-180°, +180°]$ curve, whatever the range of the file ($[0, 180]$, $[-180, 0]$ or $[-180, 180]$, Section 2.8.5).

`graph_assembly.py` 3-D picture of an orientation file with the columns $\varphi, \theta, \psi, x, y, z$ (use `--positions-first` for the order $x, y, z, \varphi, \theta, \psi$). The arrows show the dipole z axis; their length is 15 % of the size of the assembly unless `--arrow` gives it in nm. (The former version guessed the column order from the values and misread valid files with an angle larger than 3.2 rad.)

The polar plots of the GUI are drawn by the GUI itself from `out_plot`.

10.4 Comparing two output files

```
python3 tests/compare_outputs.py reference.txt new.txt [--rtol 1e-6] [--all-keys]
```

(run from a clone of the GitHub repository: the `tests/` directory is a development tool and is not installed by `pip`.) The script compares two output files quantity by quantity (the summary lines such as `av = 1.007732`, in order, and the numeric tables, row by row; Fortran exponents `1.5D-03` are accepted). It returns 0 if all the differences are within the tolerance and prints each difference otherwise. It is the tool to use to compare a run of this version with a run of a former C program on the same input.

10.5 Where the side files go

The files `out_plot` and `out_arrows_gnuplot` are written in the *directory of the output file* given on the command line (they used to go to the current directory). Running the same program twice with the same output directory overwrites them; the GUI uses a new directory for each run.

Chapter 11

Changes in Version 4

11.1 Architecture

- **Installation with pip.** Py_SHS is published on PyPI as `pyshs-optics` (`pip install pyshs-optics`, Section 1.3); the code is the package `py_shs` and the programs are the commands `pyshs-optics-gui`, `pyshs-optics-hrs`, ... (Table 1.1). The former `Src/` directory and `run_gui.py` no longer exist, and there is no `Work/` directory in the installation: the GUI uses `PySHS_Work` in the home folder (or `PYSHS_WORK`, Section 9.1), and the command-line programs read and write the files you name.
- All former C sources (`HRS.c`, `SHS.c`, `sphere_SHS.c`, `cylinder_SHS.c`, `ellipsoid_SHS.c`, `cube_SHS.c`) are replaced by pure-Python / NumPy modules with the same physics and the same stdin / file protocol. The GUI launches `python -u -m py_shs.<module> ...`; no compiler is needed.
- Documented modules: HRS, SHS, sphere (NLWKB), cylinder, ellipsoid, rectangular box / cube, and the **nonlinear Mie** program (`shs_mie.py`, SciPy required).
- Shared helper modules: `shs_grid.py` (automatic grids), `shs_units.py` (units), `shs_angles.py` (angle grid) and `shs_io.py` (side files).

11.2 Numerical changes that can modify results

Change	Why	Effect on results
Tensor symmetrised in its last two indices in every module (Section 2.8.1)	the formulas assume $\beta_{ijk} = \beta_{ikj}$; modules treated a non-symmetric tensor differently	none for a symmetric tensor; a warning is printed otherwise
Gauss–Legendre rule in $\cos\theta$ (Section 2.8.2)	exact normalisation $8\pi^2$; the midpoint rule had a +0.41% bias	$\lesssim$ 0.5% on HRS/SHS; `PYSHS_THETA_RULE=midpoint` restores the former rule
Automatic grids (Section 2.8.3)	the fixed grids silently failed for large objects	none for small objects; large corrections for large ones (Figure **??**); `PYSHS_GRID=fixed` restores the former grids
Cylinder: axial integral analytic	no aliasing for long rods; 5–10 times faster	converged result for long rods
Angle grid: $N + 1$ angles over $[0, 180°]$ in all modules (Section 2.8.5)	the same N meant different things	new angle range for HRS, SHS, cylinder; `PYSHS_ANGLE_RANGE` restores the former ranges
Cylinder polar angle $\pi/2$ exactly	the C code used 1.570796	$\sim 10^{-7}$

11.3 New features

- **Units** (Section 2.8.4): `PYSHS_UNITS=rad|raw` puts every module on the same scale; `PYSHS_I_SCALE` applies to every module; the convention is printed in the log and in the output header; the GUI has a selector.
- Warnings: non-symmetric tensor, orientation file longer than N, tensor components ignored by the Mie program, grid bounds reached.
- Side files (`out_plot`, `out_arrows_gnuplot`) written next to the output file.
- New generator `script_spheroid_random.py`; the other generators and `graph_*` scripts were repaired (Chapter 10); `script_gauss2PySHS.py` has options and validation.
- HRS: rotation computed once for all angles (several times faster for `angle_scattering`); the cylinder and box programs are vectorised (about 8 times faster) and the ellipsoid program too.
- GUI: intensity-units selector, automatic removal of old run directories, polar-plot unit decided once per file.

11.4 Equivalence with the former C programs

Same physics and same input protocol, but the results are *not* bit-identical to the C programs because of the numerical changes above, and the package does not itself demonstrate the equivalence (Chapter **??**). The option `--legacy` / `PYSHS_LEGACY=1` reproduces two defects of the old HRS/SHS C code (the missing $8\pi^2$ normalisation of b_1, b_2 and a wrong pairing of the cross terms of b_{H1}, b_{H2}); since these coefficients vanish after the isotropic average (Section 2.8.7), the option changes none of the reported quantities.

Bibliography

[1] S. Roke, M. Bonn and A. V. Petukhov, "Nonlinear optical scattering: The concept of effective susceptibility," *Phys. Rev. B* **70**, 115106 (2004).

[2] P.-M. Gassin and G. Gassin, *J. Chem. Inf. Model.* **60**, 5912–5917 (2020).

[3] P.-M. Gassin et al., *J. Chem. Inf. Model.* **61**, 5719 (2021).

[4] C. F. Bohren and D. R. Huffman, *Absorption and Scattering of Light by Small Particles*, Wiley, New York (1983).

[5] J. I. Dadap, J. Shan, K. B. Eisenthal and T. F. Heinz, "Second-harmonic Rayleigh scattering from a sphere of centrosymmetric material," *Phys. Rev. Lett.* **83**, 4045 (1999).

[6] J. I. Dadap, J. Shan and T. F. Heinz, "Theory of optical second-harmonic generation from a sphere of centrosymmetric material: small-particle limit," *J. Opt. Soc. Am. B* **21**, 1328 (2004).

Acknowledgements. Support was provided by the CAMOMILS project, ANR-15-CE21-0002 and ClaysMED project, ANR-25-CE09-0672. Contributors: Dr Pierre-Marie Gassin, Dr Lotfi Boudjema, Dr Gaelle Gassin.